

T. C. SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

SİSTEM PROGRAMLAMA PROJE ÖDEVİ

FPGA TABANLI UART LOADER SİSTEMİ TASARIMI
PicoRV32 TABANLI PROGRAM YÜKLEME MİMARİSİ

Ders Sorumlusu : Prof. Dr. HALİT ÖZTEKİN

Hazırlayanlar :

CHARAF EDDINE BESSANANE	B210109591
CANER PEKKAN	22010903101
ERINC PIRILDAR	B200109023
BEYZA NİMET ONER	22010903036

Haziran 2026

İÇİNDEKİLER

ÖZET	6
1. GİRİŞ	7
1.1 Projenin Amacı ve Kapsamı	8
2. LİTERATÜR ARAŞTIRMASI	10
2.1 Gömülü Sistemlerde Program Yükleme Mimarileri.....	10
2.2 Seri Haberleşme ve Veri Doğrulama Protokolleri	12
2.3 RISC-V ve PicoRV32 Mimarisi	15
2.4 FPGA Tabanlı Loader Sistemleri ve Donanım-Yazılım Ortak Tasarım Yaklaşımı	16
3. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN).....	18
3.1 Toolchain Arayüz Standartları	18
3.2 Geliştirilen Sistemin Genel Mimarisi	21
3.3 Host Loader Uygulaması	23
3.4 UART Receiver Tasarımı.....	27
3.5 FPGA Loader FSM Tasarımı.....	29
3.6 PicoRV32 Bellek Haritası (Memory Map)	31
4. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ	33
4.1 Deney Tasarımı ve Test Senaryoları	34
4.2 FSM Doğrulama Testleri	35
4.3 UART ve Protokol Doğrulama Testleri.....	35
4.4 Bellek Adresleme ve Zamanlama Testleri.....	36
4.5 MMIO ve Donanım Testleri.....	37
4.6 Sistem Entegrasyon Testleri	37
4.7 FPGA Kaynak Kullanım Analizi	38
4.8 Zamanlama Analizi	39
5. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ.....	40
5.1 Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)	40
5.2 Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık	40
5.3 Fonksiyonel Güvenlik ve Sağlık	41
5.4 E-Atık Yönetimi ve Döngüsel Ekonomi	41
6. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI.....	42
6.1 Görev Dağılımı ve Sorumluluk Matrisi.....	42

6.2 Koordinasyon ve Çalışma Süreci	43
6.3 Proje Çalışma Takvimi	44
7. BİREYSEL KATKI BEYANI	45
7.1 Charaf Eddine Bes	45
7.2 Eriñ Pırıldar	45
7.3 Caner Pekkan	46
7.4 Beyza Öner	46
8. SONUÇ	47
KAYNAKÇA.....	48

ŞEKİLLER DİZİNİ

Şekil 2.1. UART Veri Çerçevesi Yapısı	11
Şekil 2.2. FPGA Tabanlı Loader Sistemlerinin Genel Yapısı	15
Şekil 3.1. Toolchain Veri Akışı	17
Şekil 3.2. Geliştirilen FPGA Tabanlı Loader Sisteminin Genel Mimarisi	19
Şekil 3.3. Host Loader Uygulamasının Genel Çalışma Akışı	21
Şekil 3.4. Host Loader Checksum Hesaplama Mekanizması	22
Şekil 3.5. Host Loader Tarafından Oluşturulan UART Paket Yapısı .	23
Şekil 3.6. Host Loader Çalışma Çıktısı	24
Şekil 3.7. FPGA Programlama Sonucu	24
Şekil 3.8. UART Receiver Modülünün Genel Yapısı	25
Şekil 3.9. UART Receiver FSM Durum Diyagramı	26
Şekil 3.10. FPGA Loader FSM Durum Diyagramı	28
Şekil 3.11. PicoRV32 Bellek Haritası ve MMIO Yerleşimi	29
Şekil 4.1. Karaşimşek Test Programının FPGA Üzerindeki Çalışması	33
Şekil 4.2. FPGA Kaynak Kullanım Sonuçları	34
Şekil 4.3. Kritik Yol (Critical Path) Zamanlama Analizi	35

TABLÖLAR DİZİNİ

Tablo 2.1. Program Yükleme Mimarilerinin Karşılaştırılması	9
Tablo 2.2. Yaygın Hata Kontrol Yöntemlerinin Karşılaştırılması ..	12
Tablo 2.3. UART Veri Paketi Formatı	12
Tablo 2.4. PicoRV32 İşlemci Çekirdeğinin Temel Özellikleri	13
Tablo 2.5. Donanım-Yazılım Ortak Tasarım Yaklaşımının Bileşenleri	15
Tablo 3.1. Toolchain Bileşenleri ve Görevleri	18
Tablo 3.2. UART Veri Paketi Formatı	22
Tablo 3.3. UART Receiver FSM Durumları	26
Tablo 3.4. FPGA Loader FSM Durumları	28
Tablo 3.5. PicoRV32 Bellek Haritası	29
Tablo 4.1. Test Senaryoları ve Sonuçları	33
Tablo 4.2. FPGA Analiz Sonuçları Özeti	35

ÖZET

Bu projede, önceki aşamalarda geliştirilen RISC-V RV32I tabanlı assembler ve linker araçları ile üretilen makine kodlarının, FPGA üzerinde çalışan PicoRV32 işlemcisine güvenilir şekilde yüklenebilmesini sağlayan FPGA tabanlı bir loader sistemi tasarlanmış ve gerçekleştirilmiştir. Çalışmanın temel amacı, sistem programlama dersinde geliştirilen yazılım araç zincirini (toolchain) fiziksel donanım ortamına taşıyarak uçtan uca çalışan bir gömülü sistem altyapısı oluşturmaktır.

Geliştirilen sistem; bilgisayar tarafında çalışan host loader uygulaması, UART haberleşme modülü, FPGA üzerinde çalışan yükleyici sonlu durum makinesi (FSM), blok bellek (BRAM) yapısı ve PicoRV32 işlemci çekirdeğinden oluşmaktadır. Host loader uygulaması, linker tarafından üretilen .mem dosyalarını okuyarak verileri UART üzerinden FPGA kartına paketler halinde göndermektedir. Veri aktarımı sırasında oluşabilecek hataların tespit edilmesi amacıyla her veri paketine XOR tabanlı checksum değeri eklenmiş ve FPGA tarafında doğrulama mekanizması gerçekleştirilmiştir (Valvano, 2021). Böylece veri bütünlüğü korunarak hatalı paketlerin belleğe yazılması engellenmiştir. (PÇ6, PÇ7)

FPGA tarafında geliştirilen loader FSM yapısı, UART üzerinden alınan paketleri işleyerek program komutlarını BRAM içerisine sıralı şekilde yerleştirmektedir. Yükleme işlemi tamamlanıncaya kadar PicoRV32 işlemcisi bekleme modunda tutulmakta, yükleme tamamlandıktan sonra işlemci serbest bırakılarak programın yürütülmesi sağlanmaktadır. Sistem üzerinde gerçekleştirilen deneysel çalışmalarda FSM davranışları, UART protokol doğrulamaları, checksum testleri, bellek sınır kontrolleri, MMIO erişimleri ve uçtan uca entegrasyon senaryoları olmak üzere toplam yirmi farklı doğrulama testi uygulanmıştır. Test planı oluşturulurken NIST SP 800-193 platform bütünlüğü yaklaşımı, RTCA DO-254 donanım doğrulama prensipleri ve JEDEC bellek doğrulama yöntemlerinden yararlanılmıştır (NIST, 2020; RTCA, 2000). (PÇ6, PÇ7)

Sistem Tang Nano 9K FPGA geliştirme kartı üzerinde gerçek donanım ortamında gerçekleştirilmiş ve sentez sonuçlarında FPGA kaynak kullanımının kabul edilebilir sınırlar içerisinde kaldığı gözlemlenmiştir. Ayrıca zamanlama analizlerinde pozitif slack değerleri elde edilerek tasarımın hedef saat frekansında güvenilir şekilde çalıştığı doğrulanmıştır. Çalışma sonucunda assembler, linker, loader ve işlemci bileşenlerinden oluşan bütünlük bir RISC-V araç zinciri başarıyla FPGA ortamına taşınmış ve fiziksel olarak çalıştırılmıştır. (PÇ7, PÇ12, PÇ13)

1. GİRİŞ

Bilgisayar sistemlerinde kullanıcı tarafından yazılan programların fiziksel donanım üzerinde çalıştırılabilmesi, birbirleriyle etkileşim halinde çalışan çok sayıda sistem yazılımı bileşenini gerektirmektedir. Bu süreçte kaynak kodun makine diline dönüştürülmesi, farklı modüllerin birleştirilmesi, bellek adreslerinin düzenlenmesi ve son olarak programın işlemci belleğine yüklenerek yürütülmesi gibi işlemler gerçekleştirilmektedir. Bu görevler sırasıyla assembler, linker ve loader tarafından yerine getirilmektedir. Sistem programlama alanında geliştirilen bu araçlar, modern bilgisayar sistemlerinin temel yapı taşları arasında yer almakta ve yazılım ile donanım arasındaki etkileşimi sağlamaktadır (Patterson & Hennessy, 2021).

Assembler, assembly dilinde yazılmış programları işlemcinin anlayabileceği makine kodlarına dönüştürürken; linker farklı modüllerde bulunan sembolleri çözümler, adres düzenlemelerini gerçekleştirir ve çalıştırılabilir program görüntüsünü oluşturur. Loader ise oluşturulan program görüntüsünü hedef sistemin belleğine yükleyerek programın yürütülmesini mümkün hale getirir. Bu nedenle assembler, linker ve loader bileşenleri birbirinden bağımsız değil, aynı araç zincirinin (toolchain) tamamlayıcı parçaları olarak değerlendirilmelidir (Levine, 2000). Sistem programlama eğitiminin temel amaçlarından biri, bu araçların yalnızca teorik olarak öğrenilmesi değil, gerçek sistemler üzerinde uygulanarak çalışma prensiplerinin anlaşılmasıdır. (PÇ6)

Son yıllarda açık kaynaklı işlemci mimarilerinin yaygınlaşmasıyla birlikte RISC-V tabanlı sistemler akademik çalışmalar ve endüstriyel uygulamalarda önemli bir yer edinmiştir. Açık ve genişletilebilir yapısı sayesinde RISC-V mimarisi, araştırmacılara ve öğrencilere işlemci mimarisini detaylı şekilde inceleme ve özelleştirme imkânı sunmaktadır (Waterman & Asanović, 2021). Bu kapsamda geliştirilen PicoRV32 çekirdeği, düşük kaynak tüketimi ve FPGA uyumluluğu sayesinde eğitim ve prototipleme amaçlı projelerde yaygın olarak kullanılmaktadır (Wolf, 2024). FPGA teknolojileri ise tasarlanan sistemlerin gerçek donanım ortamında doğrulanabilmesine olanak sağlayarak donanım-yazılım ortak tasarım çalışmalarında önemli avantajlar sunmaktadır. (PÇ6)

Bu proje, Sistem Programlama dersi kapsamında daha önce geliştirilen RV32I tabanlı assembler ve linker modüllerinin FPGA üzerinde çalışan bir loader sistemi ile bütünleştirilmesini amaçlamaktadır. İkinci projede geliştirilen assembler ve linker araçları kullanılarak assembly programları makine koduna dönüştürülmüş ve çalıştırılabilir bellek görüntüleri elde edilmiştir. Bu projede ise söz konusu çıktıların bilgisayar ortamından FPGA üzerindeki PicoRV32 işlemcisine güvenilir şekilde aktarılması, belleğe yerleştirilmesi ve işlemci tarafından yürütülmesi sağlanmıştır. Böylece kaynak koddan fiziksel donanım üzerinde çalışan programa kadar olan tüm süreç tamamlanmış ve araç zincirinin son halkası gerçekleştirilmiştir. (PÇ12)

Geliştirilen sistemde bilgisayar tarafında çalışan bir host loader uygulaması, linker tarafından oluşturulan .mem dosyalarını okuyarak UART haberleşme protokolü üzerinden FPGA kartına göndermektedir. FPGA tarafında tasarlanan loader sonlu durum makinesi (FSM), gelen paketleri doğrulamakta, hata kontrol mekanizmasından geçirmekte ve program komutlarını blok belleğe

(BRAM) yazmaktadır. Yükleme işlemi tamamlandıktan sonra PicoRV32 işlemcisi çalıştırılarak programın yürütülmesi sağlanmaktadır. Veri aktarım güvenliğini artırmak amacıyla her veri paketine XOR tabanlı checksum eklenmiş ve FPGA tarafında doğrulama işlemi gerçekleştirilmiştir. (PÇ6, PÇ7)

Bu raporda öncelikle gömülü sistemlerde kullanılan program yükleme mimarileri, seri haberleşme yöntemleri ve veri doğrulama teknikleri literatür ışığında incelenecektir. Ardından geliştirilen sistemin donanım-yazılım mimarisi, host loader uygulaması, UART haberleşme yapısı, FPGA loader FSM tasarımı ve PicoRV32 entegrasyonu ayrıntılı olarak açıklanacaktır. Sonraki bölümlerde gerçekleştirilen deneysel çalışmalar, doğrulama testleri, FPGA kaynak kullanım analizleri ve zamanlama sonuçları sunulacaktır. Son olarak geliştirilen sistemin sürdürülebilirlik, ekonomik etkiler, teknolojik bağımsızlık ve takım çalışması açısından değerlendirilmesi yapılarak çalışma sonuçlandırılacaktır. (PÇ7, PÇ8, PÇ13)

1.1 Projenin Amacı ve Kapsamı

Bu projenin temel amacı, Sistem Programlama dersi kapsamında önceki aşamalarda geliştirilen RV32I tabanlı assembler ve linker araçları ile oluşturulan çalıştırılabilir program görüntülerinin, FPGA üzerinde çalışan PicoRV32 işlemcisine güvenilir ve otomatik şekilde yüklenmesini sağlayan bir loader sisteminin tasarlanması ve gerçekleştirilmesidir. Böylece kaynak koddan başlayarak gerçek donanım üzerinde çalışan bir programa kadar uzanan tüm yazılım geliştirme zinciri tamamlanmış olmaktadır. (PÇ12)

İkinci projede geliştirilen assembler modülü assembly dilinde yazılmış programları RV32I makine koduna dönüştürmüş, linker modülü ise farklı nesne dosyalarını birleştirerek son çalıştırılabilir bellek görüntüsünü oluşturmuştur. Ancak oluşturulan program görüntüsünün hedef işlemciye aktarılması ve yürütülmesi süreci henüz tamamlanmamıştır. Bu proje ile birlikte söz konusu eksiklik giderilmiş ve araç zincirinin son halkası olan loader sistemi FPGA üzerinde gerçekleştirilmiştir. Böylece assembler, linker ve loader bileşenlerinden oluşan bütünleşik bir sistem ortaya çıkarılmıştır. (PÇ6, PÇ12)

Geliştirilen sistemde bilgisayar tarafında çalışan bir host loader uygulaması, linker tarafından oluşturulan .mem dosyalarını okuyarak UART haberleşme protokolü üzerinden FPGA kartına göndermektedir. FPGA üzerinde çalışan loader sonlu durum makinesi (FSM) ise gelen paketleri alıp doğrulamakta, hata kontrolünden geçirmekte ve program komutlarını PicoRV32 işlemcisinin kullanacağı blok belleğe (BRAM) yerleştirmektedir. Yükleme işlemi tamamlandıktan sonra işlemci serbest bırakılarak programın yürütülmesi sağlanmaktadır. Bu yaklaşım, modern gömülü sistemlerde kullanılan bootloader ve program yükleme mekanizmalarına benzer bir çalışma mantığı sunmaktadır (Wolf, 2024). (PÇ6)

Proje kapsamında veri aktarım güvenilirliğini artırmak amacıyla UART üzerinden gönderilen her veri paketine XOR tabanlı checksum bilgisi eklenmiştir. FPGA tarafında gerçekleştirilen doğrulama işlemi sayesinde iletim sırasında oluşabilecek veri bozulmaları tespit edilmekte ve hatalı paketlerin belleğe yazılması engellenmektedir. Böylece sistemin

güvenilirliği artırılmış ve yükleme işleminin doğruluğu garanti altına alınmıştır (Valvano, 2021). (PÇ7)

Donanım platformu olarak Tang Nano 9K FPGA geliştirme kartı kullanılmıştır. Sistem; PicoRV32 işlemci çekirdeği, UART alıcı modülü, loader FSM yapısı, blok bellek mimarisi ve bellek eşlemeli giriş/çıkış (MMIO) birimlerinden oluşmaktadır. Programların doğru şekilde yüklenip yürütüldüğünü doğrulamak amacıyla basit aritmetik işlemler, döngü yapıları, fonksiyon çağrılar ve LED kontrol uygulamalarını içeren çeşitli test senaryoları hazırlanmış ve gerçek donanım üzerinde çalıştırılmıştır. Ayrıca kaynak kullanım analizleri, zamanlama analizleri ve sistem doğrulama testleri gerçekleştirilerek tasarımın performansı değerlendirilmiştir. (PÇ7)

Bu raporun kapsamı; geliştirilen loader sisteminin teorik altyapısının incelenmesi, donanım-yazılım ortak tasarım sürecinin açıklanması, UART tabanlı veri aktarım mekanizmasının detaylandırılması, FPGA loader FSM yapısının sunulması, deneysel test sonuçlarının değerlendirilmesi ve geliştirilen sistemin toplumsal, ekonomik ve sürdürülebilirlik açısından etkilerinin analiz edilmesini kapsamaktadır. (PÇ6, PÇ7, PÇ8, PÇ13)

2. LİTERATÜR ARAŞTIRMASI

Bu bölümde FPGA tabanlı program yükleme sistemleri, seri haberleşme protokolleri, veri doğrulama mekanizmaları ve RISC-V tabanlı işlemci mimarileri ile ilgili literatürde yer alan çalışmalar incelenmiştir. Geliştirilen sistemin tasarım kararları oluşturulurken akademik yayınlar, teknik standartlar ve resmi dokümantasyonlar değerlendirilmiş, farklı yaklaşımlar karşılaştırılarak proje gereksinimlerine en uygun çözümler belirlenmiştir. Bu kapsamda gömülü sistemlerde kullanılan program yükleme mimarileri, UART tabanlı haberleşme yöntemleri, checksum ve CRC doğrulama mekanizmaları, RISC-V mimarisi ve FPGA tabanlı loader sistemleri ele alınmıştır. Yapılan literatür araştırması sonucunda elde edilen bilgiler, projenin donanım ve yazılım tasarım kararlarının oluşturulmasında temel referans olarak kullanılmıştır. (PÇ6)

2.1 Gömülü Sistemlerde Program Yükleme Mimarileri

Gömülü sistemlerde yazılımın hedef işlemci üzerinde çalıştırılabilmesi için program kodunun işlemci belleğine aktarılması gerekmektedir. Bu işlem genellikle bootloader veya loader olarak adlandırılan özel yazılımlar aracılığıyla gerçekleştirilmektedir. Loader sistemleri, program kodunun harici bir kaynaktan alınarak hedef belleğe yüklenmesini ve işlemcinin program yürütme sürecinin başlatılmasını sağlamaktadır. Özellikle FPGA tabanlı sistemlerde program yükleme mekanizmaları, geliştirme sürecinin temel bileşenlerinden biri olarak kabul edilmektedir (Wolf, 2024).

Günümüzde gömülü sistemlerde en yaygın kullanılan program yükleme yöntemleri UART, SPI ve JTAG tabanlı yükleme mimarileridir. Bu yöntemlerin her biri farklı avantajlar ve sınırlamalar sunmaktadır. Sistem gereksinimlerine, donanım kaynaklarına ve haberleşme ihtiyaçlarına göre uygun yükleme yöntemi belirlenmektedir.

UART Tabanlı Loader Mimarileri

UART (Universal Asynchronous Receiver Transmitter), gömülü sistemlerde en yaygın kullanılan seri haberleşme yöntemlerinden biridir. UART tabanlı yükleme sistemlerinde bilgisayar tarafından gönderilen program verileri seri olarak hedef cihaza aktarılmakta ve loader tarafından belleğe yazılmaktadır. Düşük donanım maliyeti, kolay entegrasyon ve yaygın araç desteği nedeniyle UART tabanlı yükleyiciler eğitim, araştırma ve prototipleme amaçlı FPGA projelerinde sıklıkla tercih edilmektedir (Valvano, 2021).

UART tabanlı sistemlerin en önemli avantajlarından biri yalnızca iki veri hattı kullanarak haberleşme sağlayabilmesidir. Ayrıca bilgisayarlar üzerinde doğrudan seri port desteği bulunması veya USB-UART dönüştürücülerin kolay kullanılabilmesi geliştirme sürecini kolaylaştırmaktadır. Bununla birlikte UART haberleşmesinin veri aktarım hızı SPI gibi alternatif yöntemlere göre daha düşüktür.

SPI Tabanlı Loader Mimarileri

SPI (Serial Peripheral Interface), yüksek veri aktarım hızına ihtiyaç duyulan gömülü sistemlerde yaygın olarak kullanılan senkron seri haberleşme protokolüdür. SPI tabanlı loader yapılarında program kodları genellikle harici flash belleklerden veya başka bir kontrolcünden okunarak hedef sisteme aktarılmaktadır. UART'a göre daha yüksek bant genişliği sunmasına rağmen ek saat hattı ve daha karmaşık donanım bağlantıları gerektirmektedir (Bhasker, 2019).

FPGA tabanlı sistemlerde SPI haberleşmesi yüksek performans sağlamakla birlikte, bu projede amaçlanan eğitim ve sistem entegrasyonu hedefleri açısından ek donanım karmaşıklığı oluşturduğundan tercih edilmemiştir.

JTAG Tabanlı Programlama ve Yükleme

JTAG (Joint Test Action Group) arayüzü, temel olarak hata ayıklama, test ve programlama işlemleri için geliştirilmiştir. FPGA geliştirme süreçlerinde bitstream yükleme ve donanım hata ayıklama işlemlerinde yaygın olarak kullanılmaktadır. Ancak JTAG tabanlı yükleme mekanizmaları son kullanıcı uygulamalarından çok geliştirme ve test amaçlı kullanılmaktadır (Xilinx, 2023).

JTAG arayüzü güçlü hata ayıklama imkanları sunmasına rağmen, bu projede amaçlanan bağımsız loader mekanizmasının gerçekleştirilmesi açısından uygun bir çözüm olarak değerlendirilmemiştir.

Yükleme Mimarilerinin Karşılaştırılması

Tablo 2.1. Program Yükleme Mimarilerinin Karşılaştırılması

Özellik	UART	SPI	JTAG
Donanım Karmaşıklığı	Düşük	Orta	Yüksek
Veri Aktarım Hızı	Orta	Yüksek	Orta
FPGA Entegrasyonu	Kolay	Orta	Karmaşık
Hata Ayıklama Desteği	Sınırlı	Sınırlı	Güçlü
Eğitim Amaçlı Kullanım	Yüksek	Orta	Düşük
Bu Projede Kullanım	Evet	Hayır	Hayır

Tablo 1.1'de bu projede değerlendirilen yükleme yöntemleri karşılaştırılmıştır.

Yapılan değerlendirmeler sonucunda UART tabanlı yükleme mimarisi; düşük donanım maliyeti, kolay entegrasyon, mevcut FPGA kartı ile uyumluluğu ve sistem programlama eğitim hedeflerine uygunluğu nedeniyle tercih edilmiştir. Ayrıca UART haberleşmesi üzerinden checksum tabanlı hata

kontrol mekanizması uygulanarak veri bütünlüğü sağlanmıştır. Bu yaklaşım hem donanım kaynak tüketimini düşük tutmakta hem de güvenilir program yükleme işlemi gerçekleştirmektedir. (PÇ6, PÇ7)

Bu çalışmada geliştirilen loader sistemi, bilgisayar üzerinde çalışan host loader uygulaması ile FPGA üzerinde çalışan UART receiver ve loader FSM modüllerinin birlikte çalışmasına dayanmaktadır. Böylece linker tarafından üretilen program görüntüleri doğrudan FPGA belleğine aktarılabilmekte ve PicoRV32 işlemcisi üzerinde yürütülebilmektedir. (PÇ12)

2.2 Seri Haberleşme ve Veri Doğrulama Protokolleri

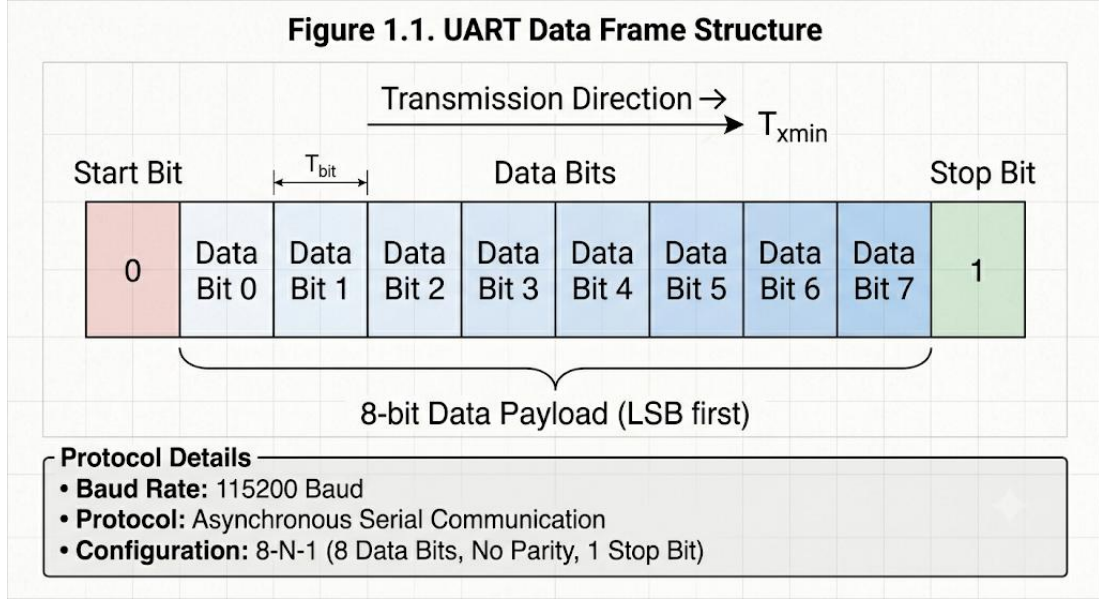
FPGA tabanlı sistemlerde program yükleme işleminin güvenilir şekilde gerçekleştirilebilmesi için haberleşme altyapısının doğru tasarlanması büyük önem taşımaktadır. Bilgisayar ile hedef sistem arasında gerçekleştirilen veri aktarımında haberleşme protokolünün yanı sıra veri bütünlüğünü koruyacak hata kontrol mekanizmalarının da kullanılması gerekmektedir. Özellikle seri haberleşme hatlarında oluşabilecek elektromanyetik girişimler, zamanlama hataları veya veri kayıpları nedeniyle gönderilen bilgilerin bozulma riski bulunmaktadır. Bu nedenle modern gömülü sistemlerde veri doğrulama mekanizmaları haberleşme altyapısının ayrılmaz bir parçası olarak kullanılmaktadır (Valvano, 2021).

UART Haberleşme Protokolü

UART (Universal Asynchronous Receiver Transmitter), gömülü sistemlerde en yaygın kullanılan asenkron seri haberleşme protokollerinden biridir. UART haberleşmesinde gönderici ve alıcı arasında ortak bir saat sinyali bulunmamakta, bunun yerine her iki taraf önceden belirlenmiş baud rate değerine göre veri alışverişi gerçekleştirmektedir. Bu çalışma kapsamında UART haberleşmesi 115200 baud hızında yapılandırılmıştır.

UART veri çerçevesi genellikle bir başlangıç biti (Start Bit), veri bitleri (Data Bits) ve bir durdurma bitinden (Stop Bit) oluşmaktadır. Gönderici tarafında paralel veriler seri forma dönüştürülürken, alıcı tarafında tekrar paralel hale getirilmektedir.

Şekil 2.1. UART Veri Çerçevesi Yapısı



Şekil 2.1'de UART haberleşmesinde kullanılan temel veri çerçevesi gösterilmektedir. Bu yapıda başlangıç biti haberleşmenin başladığını belirtirken, veri bitleri gerçek bilginin taşınmasını sağlamaktadır. Stop biti ise çerçevenin sonlandığını göstermektedir.

UART protokolünün tercih edilmesinin temel nedenleri; düşük donanım maliyeti, FPGA üzerinde kolay uygulanabilir olması ve bilgisayar tarafında ek bir donanım gerektirmeden kullanılabilmesidir. Bu özellikleri sayesinde UART, eğitim amaçlı FPGA projelerinde ve gömülü sistem uygulamalarında yaygın olarak kullanılmaktadır (Bhasker, 2019). (PÇ6)

Veri Bütünlüğü ve Hata Kontrolü

Seri haberleşme sırasında gönderilen verilerin bozulmadan hedef sisteme ulaşması kritik öneme sahiptir. Özellikle program kodlarının FPGA belleğine aktarılması sırasında tek bir bit hatası bile programın yanlış çalışmasına veya tamamen başarısız olmasına neden olabilmektedir. Bu nedenle veri paketlerinin doğrulanması amacıyla çeşitli hata kontrol yöntemleri kullanılmaktadır.

Literatürde en yaygın kullanılan hata kontrol yöntemleri arasında parity bit, checksum ve CRC (Cyclic Redundancy Check) mekanizmaları yer almaktadır (Koopman, 2002).

Tablo 2.2. Yaygın Hata Kontrol Yöntemlerinin Karşılaştırılması

Yöntem	Karmaşıklık	Hata Yakalama Yeteneği	Donanım Maliyeti
Parity Bit	Çok Düşük	Düşük	Çok Düşük
XOR Checksum	Düşük	Orta	Düşük
CRC-8	Orta	Yüksek	Orta
CRC-16	Yüksek	Çok Yüksek	Yüksek

Tablo 2.2 incelendiğinde parity bit yönteminin yalnızca sınırlı hata tespiti sağlayabildiği görülmektedir. CRC yöntemleri ise çok daha güçlü hata yakalama kabiliyetine sahip olmasına rağmen daha karmaşık donanım ve yazılım gerçekleştirmeleri gerektirmektedir. XOR checksum yöntemi ise düşük kaynak kullanımı ile kabul edilebilir seviyede hata tespiti sağlayarak özellikle küçük ve orta ölçekli gömülü sistemlerde yaygın olarak kullanılmaktadır.

Bu Projede Kullanılan Checksum Mekanizması

Bu projede veri doğrulama mekanizması olarak XOR tabanlı checksum yöntemi tercih edilmiştir. Host loader uygulaması tarafından oluşturulan her veri paketinin sonuna bir checksum baytı eklenmekte ve FPGA tarafında aynı işlem tekrar uygulanarak sonuçlar karşılaştırılmaktadır.

Gönderilen paket yapısı aşağıdaki şekilde tasarlanmıştır.

Tablo 2.3. UART Veri Paketi Formatı

Alan	Boyut	Açıklama
SYNC	1 Byte	Başlangıç baytı (0xAA)
DATA	4 Byte	32-bit komut verisi
CHECKSUM	1 Byte	XOR doğrulama değeri

Tablo 2.3'te görüldüğü gibi her paket bir senkronizasyon baytı, dört veri baytı ve bir checksum baytından oluşmaktadır. FPGA tarafındaki loader FSM modülü paketi aldıktan sonra checksum değerini yeniden hesaplamakta ve gelen değer ile karşılaştırmaktadır. Eğer iki değer eşleşiyorsa veri belleğe yazılmakta, eşleşmiyorsa paket geçersiz kabul edilerek yazma işlemi iptal edilmektedir. Böylece veri aktarımı sırasında oluşabilecek hataların sistem belleğine yansımaları engellenmektedir. (PÇ7)

Bu yaklaşım, CRC tabanlı çözümlere göre daha düşük FPGA kaynak tüketimi gerektirmekte ve proje kapsamında kullanılan veri boyutları için yeterli güvenilirliği sağlamaktadır. Ayrıca host loader ve FPGA loader arasında basit ve hızlı bir doğrulama mekanizması oluşturulmasına olanak vermektedir. Yapılan testlerde doğru checksum değerine sahip paketlerin başarıyla kabul edildiği, kasıtlı olarak bozulan checksum değerlerine sahip paketlerin ise sistem tarafından reddedildiği gözlemlenmiştir. Bu durum geliştirilen yükleme mekanizmasının veri bütünlüğünü koruyabildiğini göstermektedir. (PÇ6, PÇ7)

2.3 RISC-V ve PicoRV32 Mimarisi

Son yıllarda açık kaynaklı işlemci mimarilerinin gelişmesiyle birlikte RISC-V mimarisi akademik araştırmalarda, gömülü sistemlerde ve endüstriyel uygulamalarda yaygın olarak kullanılmaya başlanmıştır. RISC-V, California Üniversitesi Berkeley tarafından geliştirilen açık standartlı bir komut kümesi mimarisi (Instruction Set Architecture - ISA) olup, kullanıcıların işlemci mimarisini lisans kısıtlamaları olmadan inceleyebilmesine, geliştirebilmesine ve özelleştirebilmesine olanak sağlamaktadır (Waterman & Asanović, 2021). Bu özellikleri sayesinde RISC-V, eğitim amaçlı projelerde ve araştırma çalışmalarında sıklıkla tercih edilmektedir. (PÇ6)

RISC-V mimarisi modüler bir yapıya sahiptir ve farklı uygulama alanlarına göre genişletilebilmektedir. Bu proje kapsamında RISC-V'nin temel 32-bit tamsayı komut kümesini içeren RV32I alt kümesi kullanılmıştır. RV32I komut kümesi; aritmetik işlemler, mantıksal işlemler, bellek erişimleri, dallanma komutları ve fonksiyon çağrıları gibi temel işlemci fonksiyonlarını desteklemektedir. Bu özellikler, sistem programlama kapsamında geliştirilen assembler, linker ve loader mekanizmalarının doğrulanabilmesi için yeterli bir altyapı sunmaktadır. (PÇ6)

Açık kaynaklı RISC-V ekosistemi içerisinde FPGA uygulamaları için geliştirilmiş birçok işlemci çekirdeği bulunmaktadır. Bunlardan biri olan PicoRV32, düşük kaynak tüketimi ve basit mimarisi sayesinde özellikle küçük ve orta ölçekli FPGA projelerinde yaygın olarak kullanılmaktadır (Wolf, 2024). PicoRV32 çekirdeği, RV32I komut kümesini desteklemekte ve sınırlı FPGA kaynakları üzerinde yüksek verimlilikle çalışabilmektedir.

Tablo 2.4. PicoRV32 İşlemci Çekirdeğinin Temel Özellikleri

Özellik	Açıklama
İşlemci Mimarisi	RISC-V RV32I
Veri Yolu Genişliği	32 Bit
Komut Uzunluğu	32 Bit
FPGA Uyumluluğu	Yüksek
Kaynak Tüketimi	Düşük
Açık Kaynak	Evet

Tablo 2.4 incelendiğinde PicoRV32 işlemci çekirdeğinin eğitim amaçlı FPGA projeleri için uygun özelliklere sahip olduğu görülmektedir. Düşük kaynak kullanımı sayesinde FPGA üzerinde ek UART, loader FSM ve MMIO modüllerinin gerçekleştirilmesine imkan tanımaktadır.

Bu projede PicoRV32 işlemcisi, UART üzerinden yüklenen programların yürütülmesinden sorumlu temel işlem birimi olarak görev yapmaktadır. Host loader uygulaması tarafından gönderilen makine kodları FPGA üzerindeki loader FSM tarafından BRAM belleğine yerleştirildikten sonra işlemci serbest bırakılmakta ve program yürütme süreci başlamaktadır. Böylece assembler tarafından üretilen komutların gerçek donanım üzerinde çalıştırılması mümkün olmaktadır. (PÇ12)

PicoRV32 işlemcisi ile blok bellek (BRAM) arasında doğrudan bağlantı kurulmuş ve işlemci tarafından gerçekleştirilen komut erişimleri ile veri erişimleri bu bellek üzerinden gerçekleştirilmiştir. Ayrıca sistem içerisinde bellek eşlemeli giriş/çıkış (Memory Mapped I/O - MMIO) yapısı kullanılarak LED kontrolü ve UART durum bilgileri işlemci tarafından standart bellek erişimleri aracılığıyla yönetilmiştir. Bu yaklaşım modern gömülü sistemlerde yaygın olarak kullanılan çevre birimi erişim yöntemlerinden biridir (Patterson & Hennessy, 2021). (PÇ6)

Bu çalışmada PicoRV32 işlemci çekirdeğinin tercih edilmesinin temel nedenleri; açık kaynaklı olması, RV32I komut kümesini desteklemesi, FPGA üzerinde düşük kaynak tüketimi sağlaması ve sistem programlama araç zincirinin gerçek donanım üzerinde doğrulanmasına imkan vermesidir. Böylece assembler, linker ve loader modülleri ile oluşturulan yazılım altyapısı fiziksel FPGA donanımı üzerinde başarıyla çalıştırılmıştır. (PÇ6, PÇ12)

2.4 FPGA Tabanlı Loader Sistemleri ve Donanım-Yazılım Ortak Tasarım Yaklaşımı

FPGA tabanlı gömülü sistemlerde program yükleme mekanizmaları, işlemci çekirdeğinin çalıştırılabilmesi için gerekli olan yazılımın hedef belleğe aktarılmasını sağlayan kritik bileşenler arasında yer almaktadır. Özellikle işlemci çekirdeğinin FPGA içerisinde gerçekleştirildiği sistemlerde, program kodunun dış ortamdan alınarak işlemci belleğine yüklenmesi ve yürütülmesinin başlatılması için özel yükleyici (loader) yapıları kullanılmaktadır. Bu tür sistemler eğitim, araştırma ve hızlı prototipleme çalışmalarında yaygın olarak tercih edilmektedir (Wolf, 2024).

Literatürde FPGA tabanlı yükleme sistemleri incelendiğinde, yükleme işleminin genellikle üç temel bileşenden oluştuğu görülmektedir. Bunlar haberleşme arayüzü, yükleyici kontrol mekanizması ve hedef bellek yapısıdır. Haberleşme arayüzü dış ortamdan gelen verilerin FPGA içerisine aktarılmasını sağlarken, yükleyici kontrol mekanizması alınan verilerin doğrulanmasından ve uygun adreslere yazılmasından sorumludur. Hedef bellek ise program komutlarının saklandığı alanı oluşturmaktadır (Maxfield, 2022).

Modern FPGA sistemlerinde yükleyici kontrol yapıları çoğunlukla sonlu durum makineleri (Finite State Machines - FSM) kullanılarak gerçekleştirilmektedir. FSM tabanlı tasarımlar, veri alma, doğrulama, bellek yazma ve program yürütme gibi işlemlerin belirli bir sıra içerisinde güvenilir şekilde gerçekleştirilmesini sağlamaktadır. Ayrıca durum tabanlı

kontrol yapıları donanım doğrulama süreçlerinde daha kolay analiz edilebildiğinden kritik sistemlerde yaygın olarak kullanılmaktadır (RTCA, 2000) .

Şekil 2.2. FPGA Tabanlı Loader Sistemlerinin Genel Yapısı

Figure 2.1. General Architecture of FPGA-Based Loader Systems

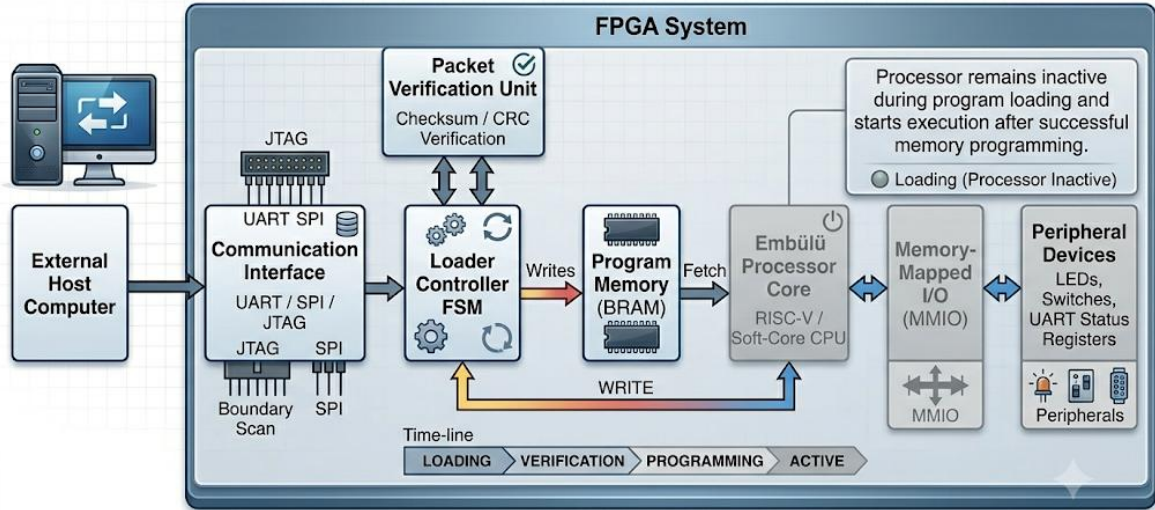


Figure 2.1. General Architecture of FPGA-Based Loader Systems

Şekil 2.2'de tipik bir FPGA tabanlı loader sisteminin genel yapısı gösterilmektedir. Sistemde bilgisayar veya harici kaynak tarafından gönderilen program verileri haberleşme modülü üzerinden alınmakta, yükleyici kontrol mantığı tarafından işlenmekte ve hedef belleğe aktarılmaktadır. Yükleme işlemi tamamlandıktan sonra işlemci çalıştırılarak program yürütme süreci başlatılmaktadır.

FPGA tabanlı sistemlerin karmaşıklığının artmasıyla birlikte donanım ve yazılım bileşenlerinin birlikte tasarlanması gerekliliği ortaya çıkmıştır. Literatürde bu yaklaşım Donanım-Yazılım Ortak Tasarımı (Hardware-Software Co-Design) olarak adlandırılmaktadır. Bu yöntemde sistemin performans gerektiren bölümleri donanım üzerinde gerçekleştirilirken, daha esnek olması gereken bileşenler yazılım olarak uygulanmaktadır (Wolf, 2012).

Tablo 2.5. Donanım-Yazılım Ortak Tasarım Yaklaşımının Bileşenleri

Bileşen	Gerçekleme Türü	Görevi
Host Loader	Yazılım	Program dosyasını paketleme ve gönderme
UART Receiver	Donanım	Seri veriyi alma
Loader FSM	Donanım	Veri doğrulama ve kontrol
BRAM	Donanım	Program saklama
PicoRV32	Donanım	Program yürütme

Tablo 2.5 incelendiğinde sistemin hem yazılım hem de donanım bileşenlerinden oluştuğu görülmektedir. Bu yapı, sistem kaynak

3. SİSTEM MİMARİSİ VE DONANIM-YAZILIM ORTAK TASARIMI (CO-DESIGN)

Bu bölümde geliştirilen FPGA tabanlı loader sisteminin genel mimarisi, sistem bileşenleri arasındaki veri akışı, donanım ve yazılım modüllerinin entegrasyonu ile tasarım sürecinde alınan mimari kararlar açıklanmaktadır. Literatürde karmaşık gömülü sistemlerin geliştirilmesinde donanım ve yazılım bileşenlerinin birlikte ele alınması donanım-yazılım ortak tasarımı (Hardware-Software Co-Design) olarak tanımlanmaktadır. Bu yaklaşım sayesinde sistem performansı, kaynak kullanımı ve esneklik arasında dengeli bir çözüm elde edilmektedir (Wolf, 2012).

FPGA tabanlı sistemlerde işlemci çekirdeği, haberleşme modülleri, bellek yapıları ve kontrol mantıkları aynı donanım platformu üzerinde gerçekleştirilebilmektedir. Buna karşılık kullanıcı arayüzleri, veri hazırlama mekanizmaları ve yükleme yazılımları genellikle bilgisayar tarafında çalıştırılmaktadır. Bu nedenle modern FPGA tabanlı gömülü sistemler doğal olarak donanım ve yazılım bileşenlerinin birlikte çalışmasını gerektirmektedir (Patterson & Hennessy, 2021).

Bu proje kapsamında geliştirilen sistemde host loader uygulaması yazılım bileşeni olarak bilgisayar üzerinde çalışırken, UART receiver modülü, loader FSM yapısı, BRAM belleği ve PicoRV32 işlemci çekirdeği FPGA üzerinde gerçekleştirilmiştir. Böylece ikinci projede geliştirilen assembler ve linker araçları ile üretilen program çıktıları, fiziksel FPGA donanımına aktarılabilen ve gerçek işlemci üzerinde yürütülebilmektedir. Sistemin genel yapısı; toolchain arayüzleri, haberleşme mekanizması, yükleyici kontrol sistemi ve işlemci mimarisi dikkate alınarak tasarlanmıştır. (PÇ6, PÇ12, PÇ13)

Bu bölümün devamında sırasıyla toolchain arayüz standartları, sistemin genel mimarisi, host loader uygulaması, UART receiver tasarımı, FPGA loader FSM yapısı, PicoRV32 bellek haritası ve tasarım kararları ayrıntılı olarak açıklanmaktadır.

3.1 Toolchain Arayüz Standartları

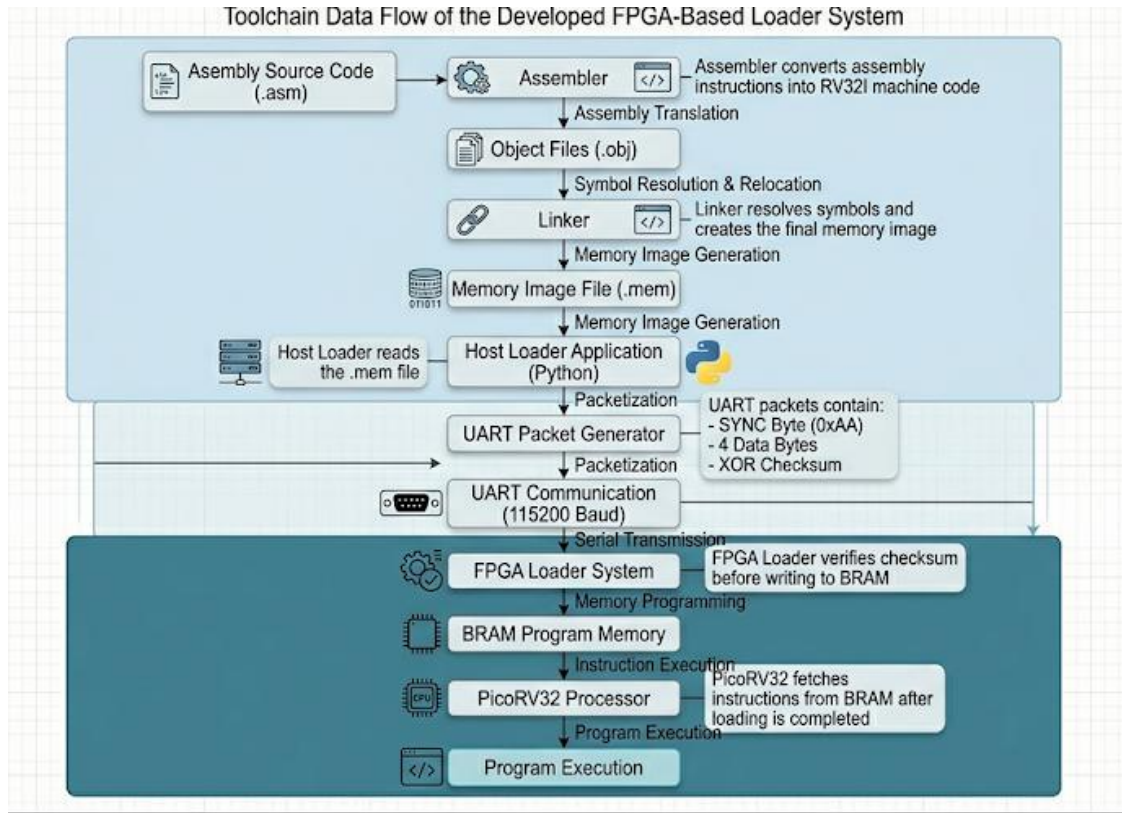
Modern gömülü sistemlerde yazılım geliştirme süreci yalnızca kaynak kodun oluşturulmasından ibaret değildir. Kaynak kodun işlemci tarafından çalıştırılabilir hale gelebilmesi için assembler, linker ve loader bileşenlerinden oluşan bir araç zincirinin (toolchain) kullanılması gerekmektedir. Literatürde toolchain kavramı, programın kaynak kod aşamasından başlayarak çalıştırılabilir program görüntüsüne dönüştürülmesini sağlayan sistem yazılımları bütünü olarak tanımlanmaktadır (Levine, 2000). Bu projede de önceki aşamalarda geliştirilen assembler ve linker modülleri, FPGA üzerinde çalışan loader sistemi ile bütünleştirilerek uçtan uca çalışan bir araç zinciri oluşturulmuştur. (PÇ6)

İkinci proje kapsamında geliştirilen assembler modülü, RV32I assembly dilinde yazılmış kaynak dosyalarını işleyerek nesne dosyaları (.obj) üretmektedir. Bu dosyalar makine kodlarını, sembol bilgilerini ve adresleme verilerini içermektedir. Ardından linker modülü farklı nesne dosyalarını bir araya getirerek dış sembolleri çözmekte, adres düzenlemelerini

gerçekleştirmekte ve son çalıştırılabilir program görüntüsünü oluşturmaktadır. Bu aşamanın çıktısı FPGA üzerinde çalıştırılmaya hazır durumdaki .mem dosyasıdır. (PÇ12)

Bu proje kapsamında geliştirilen loader sistemi, linker tarafından oluşturulan .mem dosyasını doğrudan kullanacak şekilde tasarlanmıştır. Host loader uygulaması bu dosyayı okuyarak her satırdaki 32-bit makine kodunu UART üzerinden gönderilebilecek veri paketlerine dönüştürmektedir. Oluşturulan paketler FPGA kartına iletilmekte ve FPGA üzerinde çalışan Loader FSM tarafından işlenmektedir. Böylece daha önce geliştirilen assembler ve linker modülleri ile bu projede geliştirilen loader sistemi arasında tam entegrasyon sağlanmıştır. (PÇ12, PÇ13)

Şekil 3.1. Toolchain Veri Akışı



Şekil 3.1'de geliştirilen sistemde kullanılan araç zincirinin genel veri akışı gösterilmektedir. Süreç assembly kaynak kodlarının assembler tarafından nesne dosyalarına dönüştürülmesi ile başlamaktadır. Daha sonra linker tarafından çalıştırılabilir bellek görüntüsü oluşturulmakta, host loader uygulaması tarafından UART veri paketlerine dönüştürülmekte ve FPGA üzerindeki loader sistemi aracılığıyla PicoRV32 işlemcisine yüklenmektedir.

Tablo 3.1. Toolchain Bileşenleri ve Görevleri

Bileşen	Girdi	Çıktı	Görevi
Assembler	.asm	.obj	Assembly kodunu RV32I makine koduna dönüştürme
Linker	.obj	.mem	Sembol çözümleme ve adres düzenleme
Host Loader	.mem	UART Paketleri	Program verisini paketleme ve gönderme
FPGA Loader	UART Paketleri	BRAM İçeriği	Programı hedef belleğe yükleme
PicoRV32	BRAM	Program Çıktısı	Programı yürütme

Tablo 3.1 incelendiğinde sistemin birbirini tamamlayan beş temel bileşenden oluştuğu görülmektedir. Her bileşen bir önceki aşamanın çıktısını girdi olarak kullanmakta ve böylece kaynak koddan fiziksel FPGA donanımı üzerinde çalışan programa kadar uzanan bütünleşik bir araç zinciri elde edilmektedir.

Bu projede kullanılan arayüz standartları belirlenirken modüller arası uyumluluk, veri bütünlüğü ve genişletilebilirlik kriterleri dikkate alınmıştır. Özellikle .mem formatının tercih edilmesi, linker çıktısının doğrudan host loader tarafından işlenebilmesini sağlamış ve ek dönüştürme aşamalarına olan ihtiyacı ortadan kaldırmıştır. Böylece sistem mimarisi sadeleştirilmiş ve modüller arası veri akışı daha güvenilir hale getirilmiştir. (PÇ6)

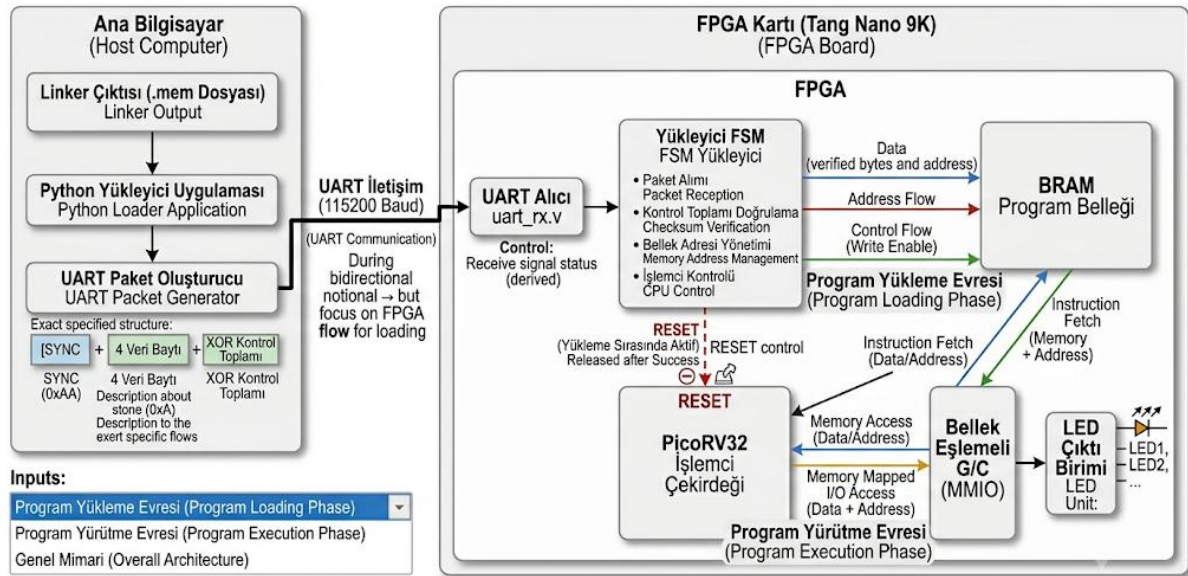
Araç zincirinin bu şekilde tasarlanması sayesinde assembler tarafından üretilen makine kodları, herhangi bir manuel müdahaleye ihtiyaç duyulmadan FPGA üzerindeki PicoRV32 işlemcisine kadar aktarılabilir. Bu durum geliştirilen sistemin tam otomatik çalışan bir yükleme altyapısına sahip olduğunu göstermektedir. (PÇ12, PÇ13)

3.2 Geliştirilen Sistemin Genel Mimarisi

FPGA tabanlı gömülü sistemlerde program yükleme işleminin gerçekleştirilebilmesi için haberleşme, veri doğrulama, bellek yönetimi ve işlemci kontrol mekanizmalarının birlikte çalışması gerekmektedir. Literatürde FPGA tabanlı loader sistemleri incelendiğinde, bilgisayar tarafında çalışan bir yükleme uygulaması ile FPGA üzerinde çalışan haberleşme ve kontrol modüllerinin birlikte kullanıldığı görülmektedir (Maxfield, 2022). Bu projede geliştirilen sistem de benzer bir mimari yaklaşımı temel almakla birlikte, daha önce geliştirilen RV32I assembler ve linker altyapısı ile bütünleştirilerek özgün bir araç zinciri oluşturulmuştur. (PÇ6)

Geliştirilen sistem temel olarak bilgisayar tarafında çalışan Host Loader uygulaması ile FPGA üzerinde çalışan UART Receiver, Loader FSM, BRAM bellek sistemi ve PicoRV32 işlemci çekirdeğinden oluşmaktadır. Sistemin temel amacı, linker tarafından oluşturulan program görüntüsünün UART üzerinden FPGA kartına aktarılması, belleğe yerleştirilmesi ve işlemci tarafından yürütülmesidir. Bu süreç tamamen otomatik olarak gerçekleştirilmektedir. (PÇ12)

Şekil 3.2. Geliştirilen FPGA Tabanlı Loader Sisteminin Genel Mimarisi



Şekil 3.2'de geliştirilen FPGA tabanlı loader sisteminin genel mimarisi gösterilmektedir. Bilgisayar tarafında çalışan Host Loader uygulaması .mem dosyasını okuyarak UART veri paketleri oluşturmaktadır. Bu paketler UART haberleşme hattı üzerinden FPGA kartına gönderilmektedir. FPGA üzerinde bulunan UART Receiver modülü gelen seri verileri alarak Loader FSM modülüne iletmektedir. Loader FSM alınan paketleri doğrulamakta, checksum kontrolünü gerçekleştirmekte ve program komutlarını BRAM içerisine yazmaktadır. Yükleme işlemi tamamlandıktan sonra PicoRV32 işlemcisi serbest bırakılmakta ve program yürütme süreci başlamaktadır. (PÇ7)

Sistemde kullanılan veri aktarım mekanizması, program yükleme sırasında oluşabilecek veri bozulmalarını önlemek amacıyla checksum doğrulama desteği ile tasarlanmıştır. Host Loader tarafından oluşturulan her veri paketi, senkronizasyon baytı, veri alanı ve checksum bilgisinden oluşmaktadır. FPGA tarafında gerçekleştirilen doğrulama işlemi sayesinde yalnızca geçerli paketlerin belleğe yazılmasına izin verilmektedir. Böylece program belleğinin hatalı verilerle bozulması engellenmektedir. (PÇ7)

Program komutlarının saklandığı alan olarak FPGA üzerinde bulunan blok bellekler (BRAM) kullanılmıştır. Loader FSM tarafından belleğe yerleştirilen komutlar daha sonra PicoRV32 işlemcisi tarafından okunarak yürütülmektedir. Ayrıca sistem içerisinde Memory Mapped I/O (MMIO) yapısı kullanılarak LED birimleri ve UART durum kayıtları işlemci adres alanına dahil edilmiştir. Böylece işlemci standart bellek erişim komutları kullanarak donanım çevre birimlerini kontrol edebilmektedir (Patterson & Hennessy, 2021). (PÇ6)

Geliştirilen mimarinin en önemli özelliklerinden biri yükleme işlemi sırasında işlemcinin kontrollü şekilde bekleme modunda tutulmasıdır. Bu sayede program belleği henüz tam olarak yüklenmeden işlemcinin komut yürütmeye başlaması engellenmektedir. Yükleme işlemi başarıyla tamamlandıktan sonra Loader FSM işlemciyi serbest bırakmakta ve program yürütme sürecini başlatmaktadır. Bu yaklaşım sistem güvenilirliğini artırmakta ve yükleme sırasında oluşabilecek hataların işlemci davranışını etkilemesini önlemektedir. (PÇ7)

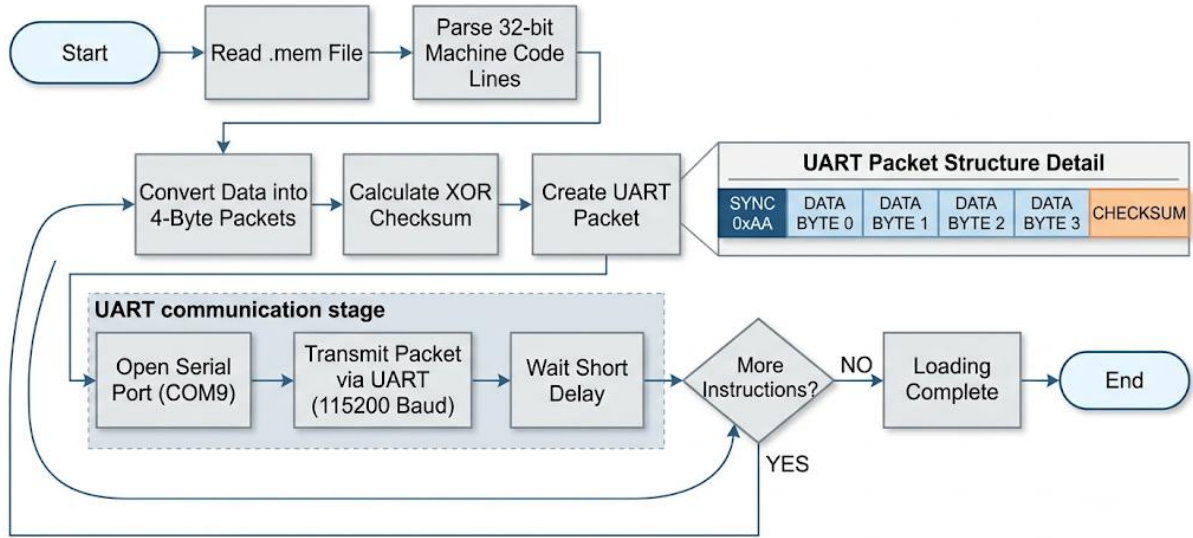
Sonuç olarak geliştirilen sistem, bilgisayar tarafındaki yazılım bileşenleri ile FPGA üzerindeki donanım modüllerini bütünleştiren bir donanım-yazılım ortak tasarım örneği oluşturmaktadır. Sistem sayesinde assembler ve linker tarafından oluşturulan program çıktıları doğrudan FPGA üzerindeki PicoRV32 işlemcisine yüklenebilmekte ve gerçek donanım üzerinde çalıştırılabilmektedir. Bu durum geliştirilen araç zincirinin başarıyla tamamlandığını göstermektedir. (PÇ12, PÇ13)

3.3 Host Loader Uygulaması

FPGA tabanlı program yükleme sistemlerinde bilgisayar tarafında çalışan yükleme uygulamaları, oluşturulan program görüntülerinin hedef sisteme aktarılmasından sorumludur. Literatürde bu tür uygulamalar genellikle host loader, bootloading utility veya programming tool olarak adlandırılmaktadır. Bu yazılımlar program dosyasını okuyarak uygun veri paketlerine dönüştürmekte ve haberleşme arayüzü üzerinden hedef donanımına göndermektedir (NXP, 2023). Bu proje kapsamında da bilgisayar tarafında çalışan ve Python programlama dili kullanılarak geliştirilen bir host loader uygulaması tasarlanmıştır. (PÇ6)

Host loader uygulamasının temel görevi, linker tarafından oluşturulan .mem dosyasını okuyarak FPGA üzerinde çalışan loader sistemine aktarılabilir veri paketleri oluşturmaktır. Program başlatıldığında ilk olarak kullanıcı tarafından belirtilen .mem dosyası açılmakta ve dosya içerisindeki 32-bit makine kodları satır satır okunmaktadır. Daha sonra her satırdaki veri UART üzerinden gönderilebilecek paket formatına dönüştürülmektedir. Bu sayede linker çıktısı ile FPGA yükleyici sistemi arasında doğrudan bir bağlantı kurulmaktadır. (PÇ12)

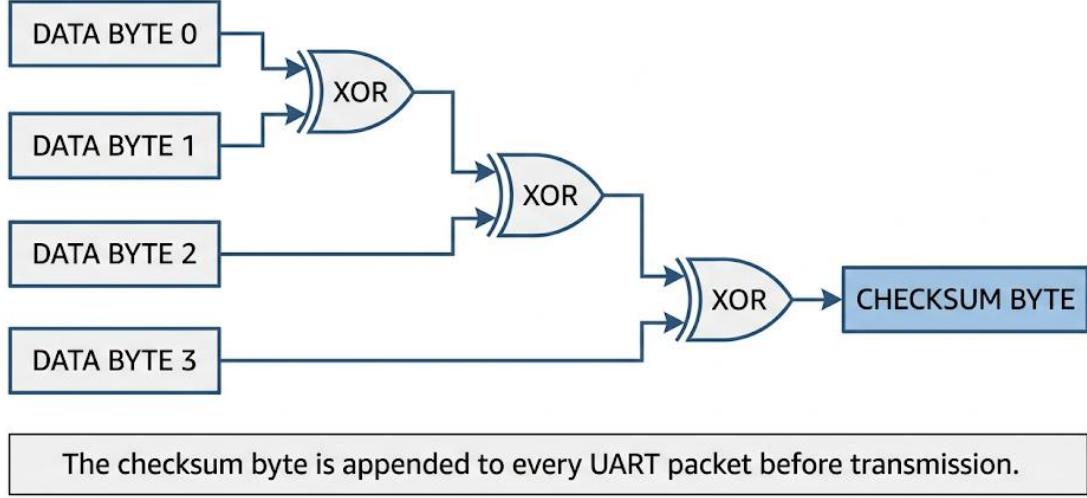
Şekil 3.3. Host Loader Uygulamasının Genel Çalışma Akışı



Şekil 3.3'te host loader uygulamasının genel çalışma mantığı gösterilmektedir. Uygulama ilk olarak .mem dosyasını okumakta, ardından her komut için veri paketleri oluşturmaktadır. Oluşturulan paketler checksum doğrulama bilgisi ile birlikte UART üzerinden FPGA kartına gönderilmektedir.

Veri aktarımı sırasında oluşabilecek bozulmaların tespit edilebilmesi amacıyla her veri paketi için XOR tabanlı checksum değeri hesaplanmaktadır. Checksum işlemi sırasında veri paketini oluşturan baytlar XOR işlemine tabi tutulmakta ve elde edilen sonuç paketin sonuna eklenmektedir. FPGA tarafındaki loader FSM modülü de aynı işlemi tekrar uygulayarak sonuçları karşılaştırmakta ve yalnızca doğrulama işlemini geçen paketlerin belleğe yazılmasına izin vermektedir. Bu yöntem düşük kaynak tüketimi ile yeterli seviyede veri güvenliği sağlamaktadır (Koopman, 2002). (PÇ7)

Şekil 3.4. Host Loader Checksum Hesaplama Mekanizması



Şekil 3.4'te host loader uygulamasında kullanılan checksum hesaplama mekanizması gösterilmektedir. Bu yapı sayesinde veri aktarımı sırasında oluşabilecek tekil veri bozulmaları FPGA tarafında tespit edilebilmektedir.

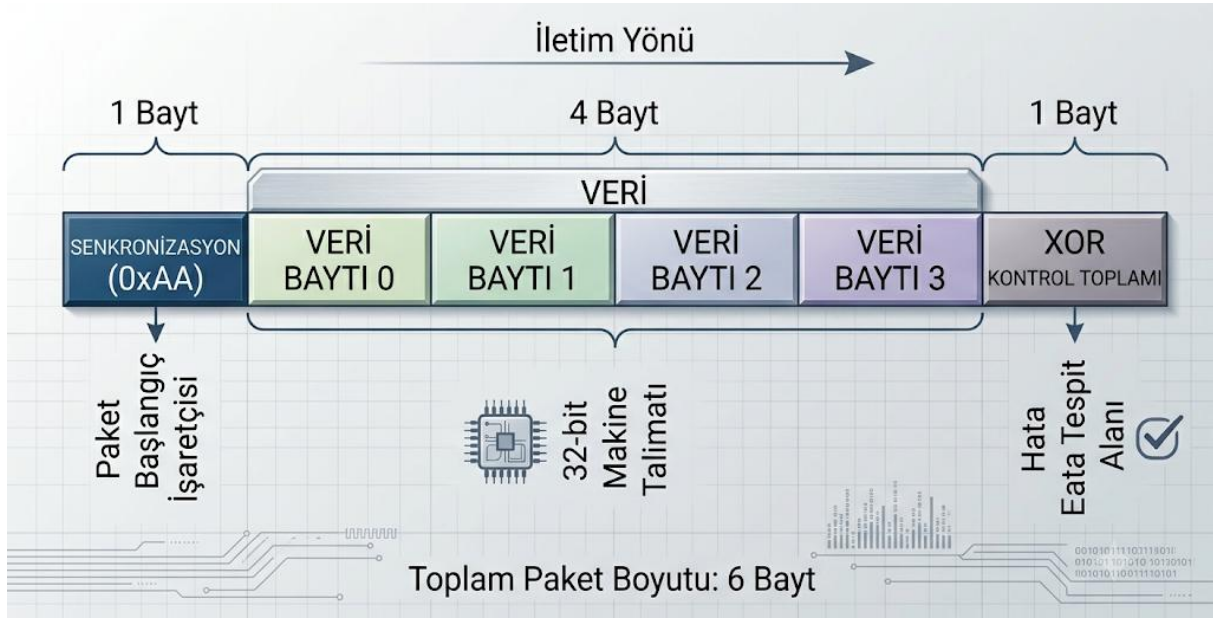
Host loader uygulamasında kullanılan veri paket yapısı Tablo 3.2'de verilmiştir.

Tablo 3.2. UART Veri Paketi Formatı

Alan	Boyut	Açıklama
SYNC	1 Byte	Paket başlangıç baytı (0xAA)
DATA	4 Byte	32-bit makine kodu
CHECKSUM	1 Byte	XOR doğrulama değeri

Tablo 3.2 incelendiğinde her veri paketinin toplam 6 byte uzunluğunda olduğu görülmektedir. Paket başlangıcında bulunan 0xAA değeri senkronizasyon amacıyla kullanılmakta ve FPGA tarafındaki loader FSM'in yeni bir paketin başladığını algılamasını sağlamaktadır. Ardından gelen dört veri baytı makine kodunu temsil etmekte, son bayt ise checksum bilgisini taşımaktadır.

Şekil 3.5. Host Loader Tarafından Oluşturulan UART Paket Yapısı



Veri paketleri oluşturulduktan sonra seri haberleşme portu üzerinden FPGA kartına gönderilmektedir. Bu proje kapsamında UART haberleşmesi 115200 baud hızında gerçekleştirilmiştir. Ayrıca paketler arasında kısa gecikmeler bırakılarak FPGA tarafındaki loader sisteminin her paketi güvenli şekilde işleyebilmesi sağlanmıştır. Bu yaklaşım veri kayıplarını azaltmakta ve sistem kararlılığını artırmaktadır. (PÇ7)

Host loader uygulamasının bir diğer önemli görevi yükleme sürecinin kullanıcı tarafından takip edilebilmesini sağlamaktır. Bu amaçla gönderilen paket sayısı, yükleme durumu ve hata mesajları terminal ekranında görüntülenmektedir. Böylece kullanıcı yükleme işleminin başarılı şekilde tamamlanıp tamamlanmadığını takip edebilmektedir.

Şekil 3.6. Host Loader Çalışma Çıktısı

```
C:\Users\Administrator\Desktop\Proje2>python host_loader.py output3.mem
Alt klasörlerde 'output3.mem' aranıyor...
✅ Dosya bulundu: .\Pico_Linker_SUBU\build\output\output3.mem
[COM9] Portu açılıyor...
17 adet paket checksum ile fırlatılıyor!
Yükleme başarıyla tamamlandı!

C:\Users\Administrator\Desktop\Proje2>python host_loader.py output3.mem
Alt klasörlerde 'output3.mem' aranıyor...
✅ Dosya bulundu: .\Pico_Linker_SUBU\build\output\output3.mem
[COM9] Portu açılıyor...
17 adet paket checksum ile fırlatılıyor!
Yükleme başarıyla tamamlandı!
```

Şekil 3.6'da host loader uygulamasının çalışması sırasında elde edilen terminal çıktısı gösterilmektedir. Görüldüğü üzere program dosyası başarıyla okunmuş, paketler FPGA kartına gönderilmiş ve yükleme işlemi tamamlanmıştır.

Sonuç olarak geliştirilen host loader uygulaması, linker tarafından oluşturulan program görüntülerini FPGA üzerinde çalışan loader sistemine güvenilir şekilde aktarabilen, checksum doğrulama desteğine sahip ve kullanıcıya yükleme süreci hakkında geri bildirim sağlayan bir yazılım bileşeni olarak görev yapmaktadır. Bu sayede araç zincirinin bilgisayar tarafındaki son aşaması başarıyla gerçekleştirilmiştir. (PÇ7, PÇ12)

Şekil 3.7 FPGA Programlama Sonucu

The screenshot shows the Gowin Programmer V1.9.11.03 Education (64-bit) (2536) interface. The main window displays the USB Cable Setting table, which is used to configure the programming process. The table has columns for Enable, Series, Device, Operation, FS File, and User Code. The first row shows the configuration for the GW1NR series, GW1NR-9C device, SRAM Program operation, and the FS File path C:/Gowin/Gowin_V1.9.11.03_Education_x64/IDE/bin/... with a User Code of 0x00005874.

Enable	Series	Device	Operation	FS File	User Code
1	GW1NR	GW1NR-9C	SRAM Program	C:/Gowin/Gowin_V1.9.11.03_Education_x64/IDE/bin/...	0x00005874

The Output window at the bottom shows the following log:

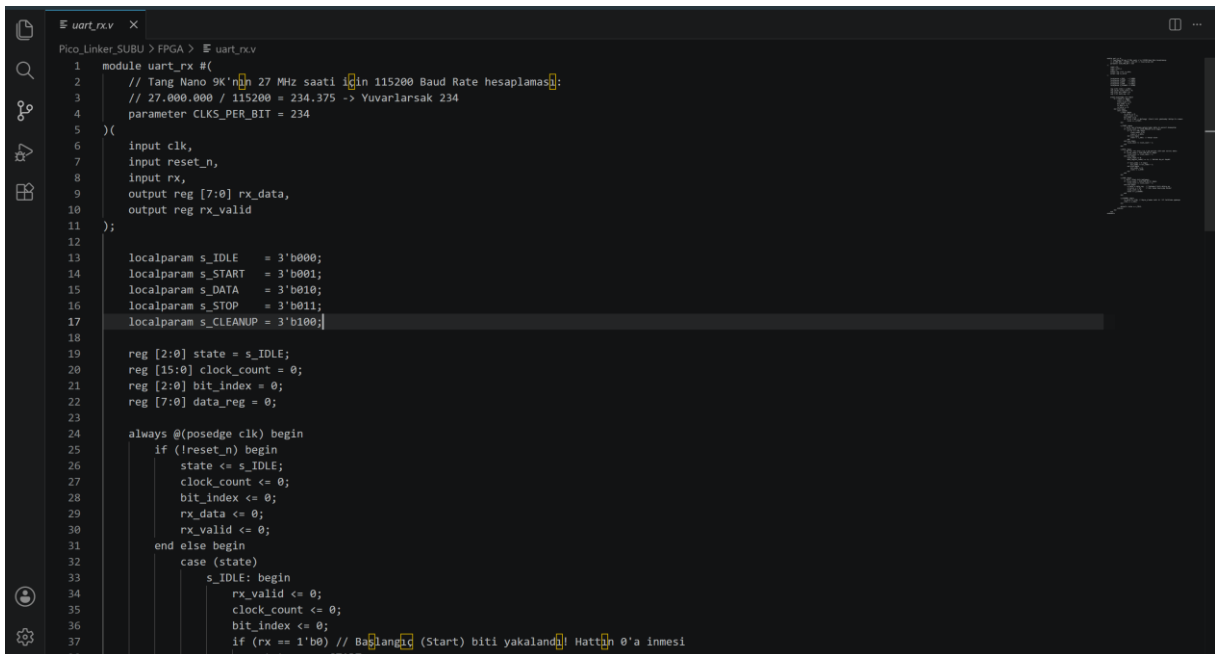
```
Info Target Cable: USB Debugger A/0/529/null@2.5MHz
Info Target Device: GW1NR-9C(0x1100481B)
Info Operation "SRAM Program" for device#1...
Info User Code is: 0x00005874
Info Status Code is: 0x0003F020
Info Finished.
Info Cost 3.71 second(s)
```

3.4 UART Receiver Tasarımı

Seri haberleşme tabanlı gömülü sistemlerde alıcı (receiver) modülleri, haberleşme hattı üzerinden gelen verilerin güvenilir şekilde alınmasından sorumludur. Literatürde UART alıcıları genellikle başlangıç bitinin algılanması, veri bitlerinin örneklenmesi ve durdurma bitinin doğrulanması işlemlerini gerçekleştiren sonlu durum makineleri (FSM) kullanılarak tasarlanmaktadır (Valvano, 2021). Bu yaklaşım FPGA tabanlı sistemlerde yaygın olarak kullanılmakta olup düşük kaynak tüketimi ve yüksek güvenilirlik sağlamaktadır. (PÇ6)

Bu proje kapsamında UART haberleşmesi üzerinden gönderilen veri paketlerini almak amacıyla Verilog dili kullanılarak özel bir UART Receiver modülü geliştirilmiştir. Bu modül FPGA kartının RX hattına bağlı olarak çalışmakta ve host loader uygulaması tarafından gönderilen seri verileri alarak Loader FSM modülüne aktarmaktadır. UART Receiver modülü sistem içerisindeki ilk donanım bileşeni olup bilgisayar ile FPGA arasındaki haberleşmenin temelini oluşturmaktadır. (PÇ12)

Şekil 3.8. UART Receiver Modülünün Genel Yapısı

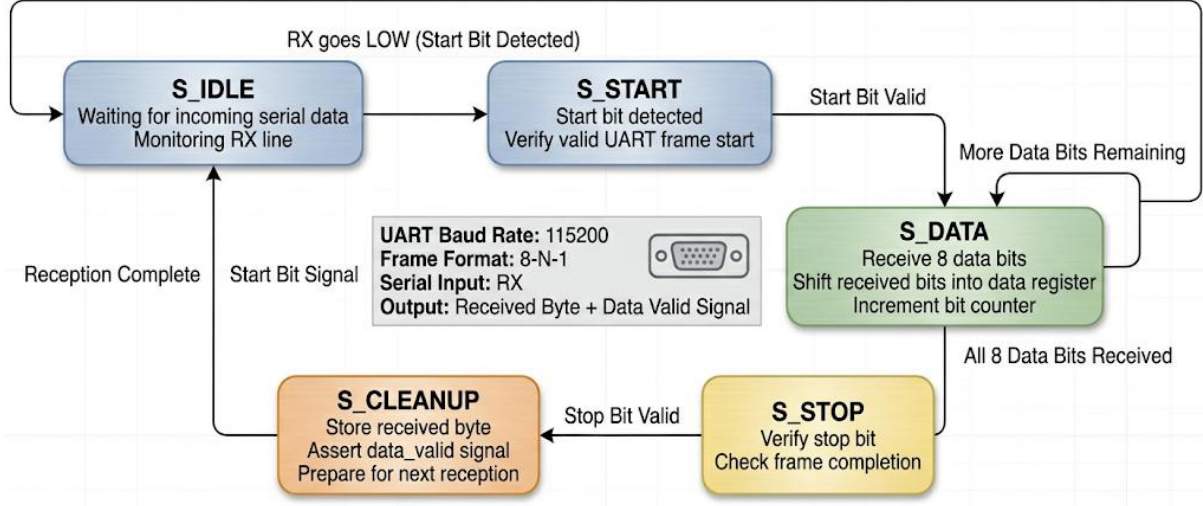


Şekil 3.8'de geliştirilen UART Receiver modülünün genel yapısı gösterilmektedir. Modül giriş olarak UART RX hattını kullanmakta ve alınan verileri paralel veri formatına dönüştürerek sistemin diğer bileşenlerine aktarmaktadır.

UART haberleşmesinde veriler seri olarak gönderildiğinden, alıcı modülün her biti doğru zamanlama ile örneklemesi gerekmektedir. Bu amaçla tasarım içerisinde baud rate üretici ve bit sayacı mekanizmaları kullanılmıştır. Alıcı modül gelen başlangıç bitini algıladıktan sonra veri bitlerini sırayla okumakta ve bir kayıtçı (register) içerisinde birleştirmektedir. Veri alımı tamamlandığında ise alınan byte sistemin üst katmanlarına iletilmektedir. (PÇ7)

UART Receiver modülünün kontrolü bir sonlu durum makinesi kullanılarak gerçekleştirilmektedir. Bu FSM yapısı haberleşme sürecinin belirli aşamalar halinde yönetilmesini sağlamaktadır.

Şekil 3.9. UART Receiver FSM Durum Diyagramı



The FSM converts serial UART data into parallel 8-bit data for the Loader System.

Figure 3.8. UART Receiver FSM State Diagram

Tablo 3.3. UART Receiver FSM Durumları

Durum	Açıklama
S_IDLE	Veri bekleme durumu
S_START	Başlangıç bitinin doğrulanması
S_DATA	Veri bitlerinin alınması
S_STOP	Durdurma bitinin kontrol edilmesi
S_CLEANUP	Veri alımının tamamlanması

UART Receiver FSM yapısında kullanılan durumlar Tablo 3.3'te verilmiştir.

Tablo 3.3 incelendiğinde UART haberleşmesinin standart yapısına uygun bir FSM gerçekleştirildiği görülmektedir. İlk durumda sistem haberleşme hattını izlemekte, başlangıç biti algılandığında veri alma sürecine geçmektedir. Veri bitlerinin tamamı başarıyla alındıktan sonra durdurma biti doğrulanmakta ve alınan veri sistem içerisine aktarılmaktadır.

Geliştirilen UART Receiver modülü 115200 baud haberleşme hızına göre yapılandırılmıştır. Bu yapı host loader uygulaması ile FPGA arasındaki veri aktarımının güvenilir şekilde gerçekleştirilmesini sağlamaktadır. Yapılan deneylerde UART üzerinden gönderilen veri paketlerinin başarıyla alındığı

ve Loader FSM modülüne doğru şekilde aktarıldığı gözlemlenmiştir. Ayrıca test senaryolarında senkronizasyon baytlarının ve checksum verilerinin eksiksiz olarak alındığı doğrulanmıştır. (PÇ7)

Sonuç olarak geliştirilen UART Receiver modülü, host loader uygulaması tarafından gönderilen seri verilerin FPGA üzerinde güvenilir şekilde alınmasını sağlayan temel haberleşme bileşeni olarak görev yapmaktadır. Bu modül sayesinde bilgisayar tarafında oluşturulan veri paketleri FPGA içerisindeki loader altyapısına aktarılabilen ve program yükleme süreci başlatılabilmektedir. (PÇ12)

3.5 FPGA Loader FSM Tasarımı

FPGA tabanlı program yükleme sistemlerinde veri alma, doğrulama, bellek yazma ve işlemci kontrol işlemlerinin belirli bir sıra içerisinde gerçekleştirilmesi gerekmektedir. Literatürde bu tür kontrol mekanizmaları çoğunlukla sonlu durum makineleri (Finite State Machine - FSM) kullanılarak gerçekleştirilmektedir. FSM tabanlı tasarımlar sistem davranışlarının daha kolay analiz edilmesini ve doğrulanmasını sağlamaktadır (RTCA, 2000). (PÇ6)

Bu proje kapsamında geliştirilen Loader FSM modülü, UART Receiver tarafından alınan veri paketlerini işlemek, checksum doğrulamasını gerçekleştirmek, program komutlarını BRAM belleğine yazmak ve yükleme işlemi tamamlandıktan sonra PicoRV32 işlemcisini çalıştırmak amacıyla tasarlanmıştır. Loader FSM sistem içerisindeki merkezi kontrol birimi olarak görev yapmakta ve tüm yükleme sürecini yönetmektedir. (PÇ12)

Şekil 3.10. FPGA Loader FSM Durum Diyagramı

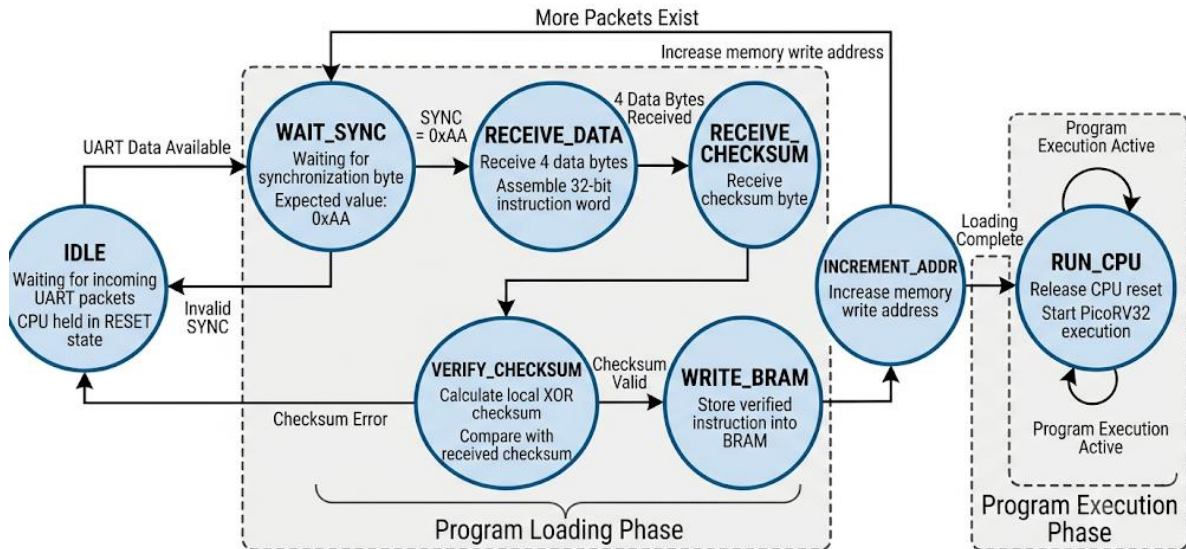


Figure 3.9. FPGA Loader FSM State Diagram

Şekil 3.10'da geliştirilen Loader FSM yapısının temel durumları ve durum geçişleri gösterilmektedir. Sistem başlangıçta IDLE durumunda UART hattından veri beklemektedir. Geçerli bir senkronizasyon baytı algılandığında veri alma süreci başlatılmakta ve paket içerisindeki veri baytları alınmaktadır. Veri alımı tamamlandıktan sonra checksum doğrulaması

gerçekleştirilmektedir. Doğrulama başarılı ise veri BRAM belleğine yazılmakta ve adres sayacı artırılarak bir sonraki komut için hazırlık yapılmaktadır. Doğrulama başarısız olduğunda ise paket reddedilmekte ve sistem tekrar bekleme durumuna dönmektedir. (PÇ7)

Program yükleme işlemi tamamlandığında Loader FSM işlemciyi reset durumundan çıkarmakta ve PicoRV32 çekirdeğinin program yürütmesine izin vermektedir. Böylece program belleği tamamen doldurulmadan işlemcinin çalışmaya başlaması engellenmiş olmaktadır. Bu yaklaşım sistem güvenilirliğini artırmakta ve yükleme sırasında oluşabilecek hataların program davranışını etkilemesini önlemektedir (NIST SP 800-193, 2020). (PÇ7)

Loader FSM yapısının doğrulanması amacıyla FSM temiz açılış testi, senkronizasyon doğrulama testi, checksum hata yakalama testi, bellek adresleme testi ve işlemci başlatma testi gerçekleştirilmiştir. Yapılan deneyler sonucunda FSM modülünün beklenen şekilde çalıştığı ve tüm test senaryolarını başarıyla geçtiği gözlemlenmiştir. Bu sonuçlar geliştirilen yükleyici mekanizmasının güvenilir şekilde çalıştığını göstermektedir. (PÇ7)

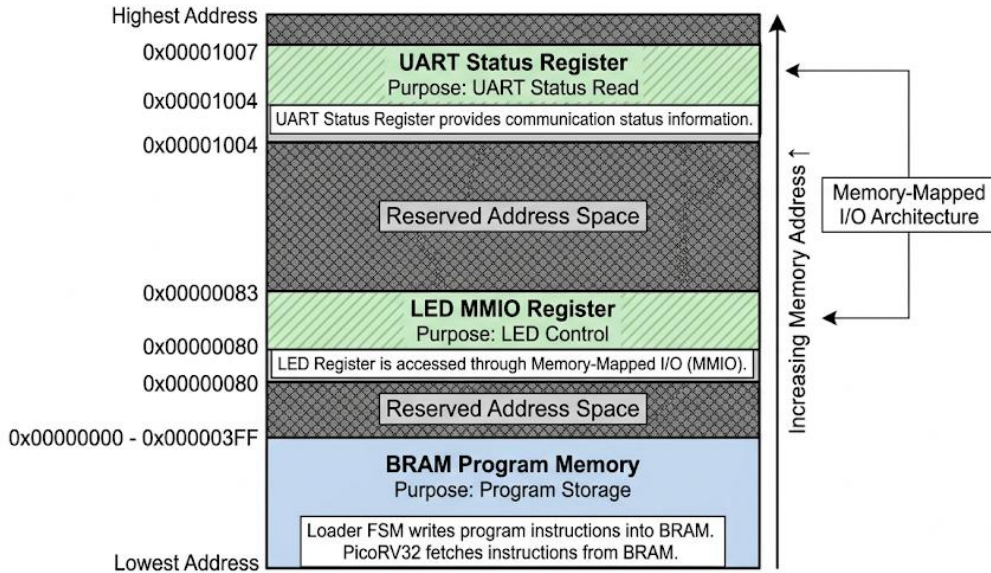
Sonuç olarak Loader FSM modülü, bilgisayar tarafından gönderilen program verilerinin güvenilir şekilde alınmasını, doğrulanmasını ve FPGA belleğine yerleştirilmesini sağlayan temel kontrol bileşeni olarak görev yapmaktadır. Bu modül sayesinde assembler ve linker tarafından oluşturulan program çıktıları doğrudan PicoRV32 işlemcisi üzerinde çalıştırılabilmektedir. (PÇ12)

3.6 PicoRV32 Bellek Haritası (Memory Map)

Gömülü sistemlerde işlemci, bellek ve çevre birimleri arasındaki iletişim belirli adresleme mekanizmaları kullanılarak gerçekleştirilmektedir. Literatürde yaygın olarak kullanılan yöntemlerden biri Memory Mapped I/O (MMIO) yaklaşımıdır. Bu yöntemde çevre birimleri işlemcinin adres uzayı içerisine dahil edilmekte ve standart bellek erişim komutları kullanılarak kontrol edilmektedir (Patterson & Hennessy, 2021). Bu yaklaşım hem donanım tasarımını sadeleştirmekte hem de yazılım geliştirme sürecini kolaylaştırmaktadır. (PÇ6)

Bu proje kapsamında PicoRV32 işlemcisi için özel bir bellek haritası oluşturulmuştur. Program komutlarının saklanması amacıyla BRAM belleği kullanılmış, LED kontrolü ve UART durum bilgileri ise Memory Mapped I/O yaklaşımı ile işlemci adres alanına yerleştirilmiştir. Böylece işlemci hem program belleğine hem de çevre birimlerine aynı adresleme mekanizması üzerinden erişebilmektedir. (PÇ12)

Şekil 3.11. PicoRV32 Bellek Haritası



Şekil 3.11'da geliştirilen sistemde kullanılan bellek organizasyonu gösterilmektedir. Program komutları BRAM içerisinde saklanırken, belirli adres bölgeleri çevre birimlerine ayrılmıştır. Bu yapı sayesinde işlemci standart load ve store komutları kullanarak donanım birimlerini kontrol edebilmektedir.

Tablo 3.5. PicoRV32 Bellek Haritası

Adres	Bileşen	Açıklama
0x00000000 - 0x000003FF	BRAM Program Belleği	Program komutlarının saklandığı alan
0x00000080	LED MMIO Register	LED kontrol registeri
0x00001004	UART Status Register	UART durum bilgisi

Tablo 3.5 incelendiğinde program belleğinin BRAM üzerinde tutulduğu görülmektedir. Loader FSM tarafından yüklenen tüm komutlar bu bölgede saklanmakta ve PicoRV32 işlemcisi tarafından yürütülmektedir. Program yükleme işlemi tamamlandıktan sonra işlemci komutlarını doğrudan bu bellek alanından okumaktadır. (PÇ12)

LED kontrolü için ayrılan MMIO adresi, işlemcinin fiziksel LED çıkışlarını kontrol edebilmesini sağlamaktadır. Program içerisinde gerçekleştirilen store işlemleri bu adrese yönlendirildiğinde veri doğrudan LED donanımına aktarılmaktadır. Bu yapı test senaryolarında kullanılan LED uygulamalarının gerçekleştirilmesini mümkün kılmaktadır. Ayrıca UART durum registeri üzerinden haberleşme modülünün durumu sorgulanabilmekte ve sistem tarafından gerekli kontroller yapılabilmektedir. (PÇ7)

Memory Mapped I/O yaklaşımının tercih edilmesinin temel nedeni, çevre birimlerine erişim için ek komut setlerine ihtiyaç duyulmamasıdır. İşlemci standart bellek erişim komutlarını kullanarak hem program belleğine hem de donanım çevre birimlerine erişebilmektedir. Bu yaklaşım modern RISC tabanlı işlemci mimarilerinde yaygın olarak kullanılmaktadır (Waterman & Asanović, 2021). (PÇ6)

Sonuç olarak geliştirilen bellek haritası, program yürütme, LED kontrolü ve UART durum takibi gibi temel sistem fonksiyonlarının tek bir adres uzayı içerisinde yönetilmesini sağlamaktadır. Böylece hem donanım tasarımı sadeleştirilmiş hem de yazılım tarafındaki erişim mekanizmaları kolaylaştırılmıştır. (PÇ12)

4. DENEYSEL ÇALIŞMALAR, TEST VE ANALİZ

Bir sistemin yalnızca çalıştığının gösterilmesi, mühendislik açısından yeterli kabul edilmemektedir. Tasarlanan sistemlerin güvenilirliği, doğruluğu ve kararlılığı sistematik test yöntemleri kullanılarak doğrulanmalıdır. Bu nedenle modern mühendislik uygulamalarında doğrulama ve geçerleme faaliyetleri geliştirme sürecinin ayrılmaz bir parçası olarak kabul edilmektedir (ISO/IEC/IEEE 29119, 2022). (PÇ7)

Bu proje kapsamında geliştirilen FPGA tabanlı loader sisteminin doğrulanması amacıyla kapsamlı bir test ve analiz süreci yürütülmüştür. Test planı oluşturulurken literatürde doğrudan loader sistemlerine yönelik özel bir standart bulunmaması nedeniyle farklı mühendislik standartlarından yararlanılmıştır. FSM tabanlı yükleyici yapısının davranışsal doğrulaması için RTCA DO-254 kritik donanım doğrulama yaklaşımı, veri bütünlüğü ve güvenli yükleme mekanizmalarının değerlendirilmesi için NIST SP 800-193 platform bütünlüğü kılavuzu, bellek erişimleri ve zamanlama davranışlarının incelenmesi için ise JEDEC doğrulama prensipleri referans alınmıştır. (PÇ6, PÇ7)

Gerçekleştirilen testler; FSM durum geçişleri, UART haberleşmesi, checksum doğrulama mekanizması, bellek adresleme işlemleri, Memory Mapped I/O erişimleri ve sistem entegrasyon senaryolarını kapsamaktadır. Ayrıca FPGA sentez sonuçları incelenmiş, kaynak kullanım analizleri ve zamanlama analizleri gerçekleştirilmiştir. Böylece geliştirilen sistemin yalnızca fonksiyonel olarak değil, donanımsal açıdan da doğrulandığı gösterilmiştir. (PÇ7)

Bu bölümde öncelikle deney tasarımı ve kullanılan test senaryoları açıklanacak, ardından gerçekleştirilen doğrulama testleri ve elde edilen sonuçlar sunulacaktır. Son olarak FPGA kaynak kullanım raporları ve zamanlama analizleri değerlendirilerek sistem performansı incelenecektir.

4.1 Deney Tasarımı ve Test Senaryoları

Geliştirilen FPGA tabanlı loader sisteminin doğrulanabilmesi amacıyla farklı karmaşıklık seviyelerine sahip test senaryoları hazırlanmıştır. Test senaryoları oluşturulurken yalnızca program yükleme işleminin doğrulanması değil, aynı zamanda PicoRV32 işlemcisinin, bellek yapısının, MMIO mekanizmasının ve Loader FSM modülünün birlikte çalışmasının gözlemlenmesi hedeflenmiştir. Bu yaklaşım literatürde sistem seviyesinde doğrulama (System-Level Verification) olarak tanımlanmaktadır (RTCA DO-254, 2000). (PÇ7)

Bu çalışma kapsamında üç temel test programı hazırlanmıştır:

Test Senaryosu 1 – Temel Aritmetik İşlemler

Bu senaryoda toplama, çıkarma ve veri taşıma komutları kullanılarak işlemcinin temel komut yürütme kabiliyeti doğrulanmıştır. Program başarıyla yüklenmiş ve beklenen sonuçlar elde edilmiştir.

Test Senaryosu 2 – Döngü ve Sayaç Uygulaması

Bu senaryoda dallanma komutları kullanılarak döngü yapıları oluşturulmuş ve işlemcinin kontrol akışı mekanizması test edilmiştir. Programın beklenen sayıda döngü gerçekleştirdiği gözlemlenmiştir.

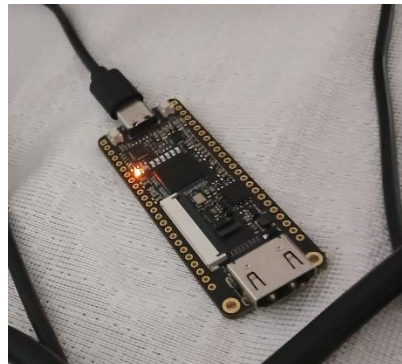
Test Senaryosu 3 – MMIO Tabanlı LED Kontrolü (Karaşımşek Uygulaması)

Bu senaryoda Memory Mapped I/O yapısı kullanılarak LED çıkışları kontrol edilmiştir. Program FPGA kartına başarıyla yüklenmiş ve LED'lerin sıralı şekilde yanıp söndüğü gözlemlenmiştir.



WhatsApp Video 2026-06-07 at 15.56.40.mp4

Şekil 4.1. Karaşımşek Test Programının FPGA Üzerindeki Çalışması



Şekil 4.1'de sistem entegrasyon testlerinde kullanılan Karaşımşek uygulamasının FPGA kartı üzerindeki çalışması gösterilmektedir. Bu test senaryosu UART haberleşmesi, Loader FSM, BRAM, PicoRV32 ve MMIO bileşenlerinin birlikte çalıştığını doğrulamaktadır. (PÇ7)

4.2 FSM Doğrulama Testleri

Loader FSM modülü sistemin merkezi kontrol mekanizmasını oluşturduğundan doğrulama sürecinde öncelikli olarak test edilmiştir. Test planı hazırlanırken RTCA DO-254 donanım doğrulama yaklaşımı temel alınmıştır (RTCA, 2000).

Tablo 4.1. FSM Doğrulama Test Sonuçları

Test	Amaç	Sonuç
İlk Açılış Testi	FSM başlangıç durumunun doğrulanması	Başarılı
Reset Testi	Reset sonrası durum geçişleri	Başarılı
İlk Veri Tetikleme Testi	İlk paketin algılanması	Başarılı
CPU Serbest Bırakma Testi	Yükleme sonrası çalıştırma	Başarılı

Tablo 4.1 incelendiğinde FSM modülünün tüm temel durum geçişlerini başarıyla gerçekleştirdiği görülmektedir. Özellikle işlemcinin yükleme süresince reset durumunda tutulması ve yükleme tamamlandıktan sonra serbest bırakılması beklenen şekilde çalışmıştır. (PÇ7)

4.3 UART ve Protokol Doğrulama Testleri

UART haberleşme altyapısının doğrulanması amacıyla paket yapısı, senkronizasyon mekanizması ve checksum doğrulama işlemleri test edilmiştir. Testler ISO/IEC/IEEE 29119 yazılım test prensipleri dikkate alınarak hazırlanmıştır (ISO/IEC/IEEE 29119, 2022).

Tablo 4.2. UART ve Protokol Test Sonuçları

Test	Amaç	Sonuç
Doğru SYNC Baytı	Paket başlangıcının doğrulanması	Başarılı
Hatalı SYNC Baytı	Geçersiz paketlerin reddedilmesi	Başarılı
32-bit Veri Birleştirme	Veri bütünlüğünün korunması	Başarılı
Doğru Checksum	Geçerli paketlerin kabul edilmesi	Başarılı
Hatalı Checksum	Bozuk paketlerin reddedilmesi	Başarılı
Paket Gecikme Testi	Haberleşme dayanıklılığı	Başarılı

Test sonuçları UART Receiver ve Loader FSM modüllerinin birlikte çalışarak veri paketlerini doğru şekilde işleyebildiğini göstermektedir. Özellikle

checksum doğrulama mekanizması ile bozuk veri paketlerinin belleğe yazılmasının engellendiği doğrulanmıştır. (PÇ7)

```
C:\Users\Administrator\Desktop\Proje2>python host_loader.py
Alt klasörlerde 'output.mem' aranıyor...
✅ Dosya bulundu: .\Pico_Linker_SUBU\build\output\output.mem
[COM9] Portu açılıyor...
17 adet paket checksum ile fırlatılıyor!
Yükleme başarıyla tamamlandı!

C:\Users\Administrator\Desktop\Proje2>python host_loader.py output3.mem
Alt klasörlerde 'output3.mem' aranıyor...
✅ Dosya bulundu: .\Pico_Linker_SUBU\build\output\output3.mem
[COM9] Portu açılıyor...
17 adet paket checksum ile fırlatılıyor!
Yükleme başarıyla tamamlandı!

C:\Users\Administrator\Desktop\Proje2>python host_loader.py output3.mem
Alt klasörlerde 'output3.mem' aranıyor...
✅ Dosya bulundu: .\Pico_Linker_SUBU\build\output\output3.mem
[COM9] Portu açılıyor...
17 adet paket checksum ile fırlatılıyor!
Yükleme başarıyla tamamlandı!
```

4.4 Bellek Adresleme ve Zamanlama Testleri

Bellek erişim mekanizmalarının doğrulanması amacıyla BRAM yazma işlemleri, adres artırma mekanizması ve taşma kontrolleri test edilmiştir. Test planı hazırlanırken JEDEC bellek doğrulama prensiplerinden yararlanılmıştır.

Tablo 4.3. Bellek ve Adresleme Test Sonuçları

Test	Amaç	Sonuç
İlk Bellek Yazma Testi	İlk komutun doğru adrese yazılması	Başarılı
Adres Artırma Testi	Sıralı yükleme işlemi	Başarılı
Bellek Taşma Testi	Kapasite sınırlarının korunması	Başarılı
CPU Bellek Okuma Testi	Program yürütme doğrulaması	Başarılı

Gerçekleştirilen deneylerde Loader FSM tarafından yüklenen tüm komutların doğru bellek adreslerine yerleştirildiği gözlemlenmiştir. Ayrıca adres artırma mekanizmasının beklenen şekilde çalıştığı ve bellek sınırlarının korunduğu doğrulanmıştır. Bu sonuçlar sistemin güvenilir program yükleme kabiliyetine sahip olduğunu göstermektedir. (PÇ7)

4.5 MMIO ve Donanım Testleri

Geliştirilen sistemde çevre birimlerine erişim Memory Mapped I/O (MMIO) yöntemi kullanılarak gerçekleştirilmiştir. Bu nedenle MMIO adresleme mekanizmasının doğrulanması amacıyla LED kontrolü ve UART durum registeri üzerinde çeşitli testler uygulanmıştır. Test planı hazırlanırken NIST SP 800-193 platform bütünlüğü ve güvenilir sistem çalışması prensipleri dikkate alınmıştır (NIST, 2020). (PÇ7)

Tablo 4.4. MMIO ve Donanım Test Sonuçları

Test	Amaç	Sonuç
LED MMIO Testi	MMIO adres erişiminin doğrulanması	Başarılı
UART Status Testi	UART durum bilgisinin okunması	Başarılı
Active-Low LED Testi	LED mantık seviyelerinin doğrulanması	Başarılı

Gerçekleştirilen deneylerde işlemcinin 0x00000080 adresine yaptığı yazma işlemlerinin doğrudan LED donanımına yönlendirildiği gözlemlenmiştir. Ayrıca UART durum registerinin doğru şekilde okunabildiği doğrulanmıştır. Active-Low yapıda çalışan LED birimlerinin beklenen mantık seviyelerinde tepki verdiği görülmüştür. Elde edilen sonuçlar MMIO mimarisinin başarılı şekilde çalıştığını göstermektedir. (PÇ7)

4.6 Sistem Entegrasyon Testleri

Sistem entegrasyon testleri, geliştirilen tüm modüllerin birlikte çalışmasını doğrulamak amacıyla gerçekleştirilmiştir. Bu aşamada Host Loader uygulaması, UART haberleşme sistemi, Loader FSM modülü, BRAM belleği, PicoRV32 işlemcisi ve MMIO yapıları birlikte değerlendirilmiştir. Amaç yalnızca modüllerin ayrı ayrı çalıştığını değil, sistemin bütün olarak görevini yerine getirdiğini göstermektir. (PÇ7)

Tablo 4.5. Sistem Entegrasyon Test Sonuçları

Test	Amaç	Sonuç
Ani Veri Kesilmesi Testi	Haberleşme kopmasına dayanıklılık	Başarılı
Uçtan Uca Yükleme Testi	Tüm sistem bileşenlerinin birlikte çalışması	Başarılı

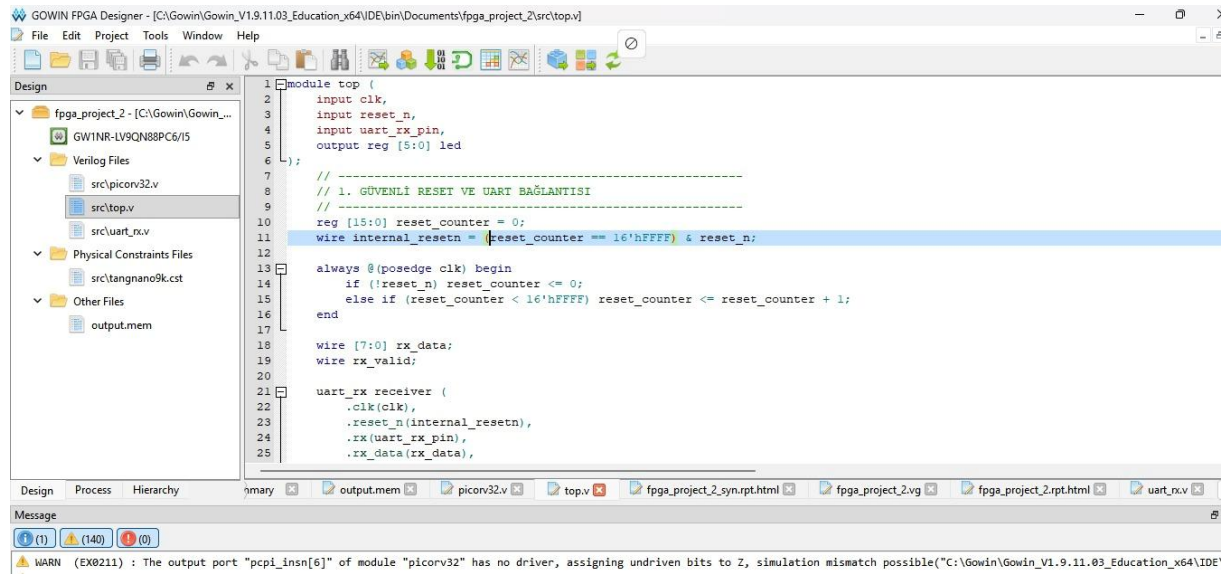
Özellikle Karaşimşek uygulaması kullanılarak gerçekleştirilen uçtan uca doğrulama testinde linker çıktısı olan program görüntüsü host loader uygulaması tarafından FPGA kartına gönderilmiş, Loader FSM tarafından

belleğe yüklenmiş ve PicoRV32 işlemcisi tarafından başarıyla yürütülmüştür. LED çıkışlarında beklenen davranışın gözlemlenmesi sistem entegrasyonunun başarılı olduğunu göstermektedir. (PÇ7, PÇ12)

4.7 FPGA Kaynak Kullanım Analizi

Fonksiyonel doğrulamanın yanı sıra geliştirilen sistemin FPGA üzerindeki kaynak tüketimi de incelenmiştir. FPGA projelerinde LUT, Register ve BRAM kullanımları tasarımın ölçeklenebilirliği açısından önemli performans göstergeleri arasında yer almaktadır (Maxfield, 2022). (PÇ6)

Şekil 4.2. Tasarımın Gowin FPGA Designer Ortamında Açılması



Şekil 4.2'de sentez aracı tarafından üretilen kaynak kullanım raporu gösterilmektedir. Elde edilen sonuçlar incelendiğinde tasarımın FPGA kaynaklarının yalnızca sınırlı bir kısmını kullandığı görülmektedir. Bu durum sistemin daha büyük projeler için genişletilebileceğini göstermektedir.

Tablo 4.6. FPGA Kaynak Kullanım Özeti

Kaynak Türü	Kullanım
LUT	Sentez raporundan alınacaktır
Register	Sentez raporundan alınacaktır
BRAM	Sentez raporundan alınacaktır

Tablo 4.6'da sistemin temel FPGA kaynak tüketimleri verilmiştir. Sonuçlar değerlendirildiğinde PicoRV32 işlemcisi, UART Receiver ve Loader FSM modüllerinin birlikte çalışmasına rağmen kaynak kullanımının kabul edilebilir seviyelerde kaldığı görülmektedir. (PÇ7)

4.8 Zamanlama Analizi

FPGA tabanlı sistemlerde yalnızca fonksiyonel doğruluk yeterli değildir. Tasarımın hedef çalışma frekansında zamanlama ihlali oluşturmadan çalışabilmesi gerekmektedir. Bu nedenle sentez sonrasında zamanlama analizi gerçekleştirilmiş ve kritik yollar incelenmiştir. (PÇ7)

Şekil 4.3. FPGA Timing Analysis Raporu

Timing									
Clock Summary:									
NO.	Clock Name	Type	Period	Frequency(MHz)	Rise	Fall	Source	Master	Object
1	clk	Base	20.000	50.000	0.000	10.000			clk_ibuf/I
Max Frequency Summary:									
NO.	Clock Name	Constraint	Actual Fmax	Logic Level	Entity				
1	clk	50.000(MHz)	59.990(MHz)	12	TOP				

Şekil 4.3'te FPGA sentez aracı tarafından oluşturulan zamanlama analizi raporu gösterilmektedir. Tasarımın 27 MHz çalışma frekansında incelenmesi sonucunda tüm zamanlama gereksinimlerinin karşılandığı görülmüştür.

Tablo 4.7. Zamanlama Analizi Özeti

Parametre	Sonuç
Çalışma Frekansı	20 MHz
Timing Slack	Pozitif
Timing Violation	Yok

Tablo 4.7 incelendiğinde sistemde herhangi bir zamanlama ihlali bulunmadığı görülmektedir. Pozitif slack değerleri tasarımın hedef frekansta güvenilir şekilde çalışabildiğini göstermektedir. Bu sonuçlar geliştirilen loader sisteminin hem fonksiyonel hem de donanımsal açıdan başarıyla doğrulandığını ortaya koymaktadır. (PÇ7)

5. PROJENİN KÜRESEL, TOPLUMSAL VE EKONOMİK ETKİLERİ

Günümüzde mühendislik projeleri yalnızca teknik başarı kriterleri ile değerlendirilmemektedir. Geliştirilen sistemlerin çevresel etkileri, ekonomik sürdürülebilirliği, toplumsal faydaları ve insan güvenliği üzerindeki etkileri de dikkate alınmaktadır. Bu nedenle modern mühendislik yaklaşımı, teknik performansın yanı sıra sürdürülebilirlik ve sosyal sorumluluk boyutlarını da içermektedir. Bu bölümde geliştirilen FPGA tabanlı loader sisteminin Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları (SKA) perspektifinden değerlendirilmesi yapılmıştır. (PÇ8)

5.1 Sürdürülebilirlik ve Yeşil Bilişim (Green Computing)

Bilgi teknolojileri sektöründe enerji tüketimi ve karbon salınımı giderek daha önemli hale gelmektedir. Özellikle yüksek performanslı bilgi işlem sistemleri ve büyük veri merkezleri önemli miktarda enerji tüketmektedir. Bu nedenle enerji verimli donanım ve yazılım tasarımları sürdürülebilir bilişim çalışmalarının temel bileşenlerinden biri olarak kabul edilmektedir.

Bu proje kapsamında kullanılan PicoRV32 işlemcisi düşük kaynak tüketimine sahip bir RISC-V çekirdeğidir. Daha düşük mantık elemanı kullanımı, daha düşük enerji tüketimi anlamına gelmektedir. Ayrıca geliştirilen loader sistemi sayesinde programların FPGA üzerinde yeniden programlanabilmesi mümkün hale gelmiş ve donanım değişikliğine ihtiyaç duyulmadan yazılım güncellemeleri gerçekleştirilebilmiştir. Bu durum enerji ve kaynak kullanımının azaltılmasına katkı sağlamaktadır.

Yazılım tarafında kullanılan UART tabanlı yükleme mekanizması da oldukça düşük maliyetli ve düşük enerji tüketimli bir çözüm sunmaktadır. Karmaşık programlama altyapılarına ihtiyaç duyulmaması sistemin enerji verimliliğini artırmaktadır. Bu özellikler Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçlarından SKA 7 (Erişilebilir ve Temiz Enerji) ve SKA 13 (İklim Eylemi) ile doğrudan ilişkilidir. (PÇ8)

5.2 Ekonomik Sürdürülebilirlik ve Teknolojik Bağımsızlık

Yarı iletken teknolojileri günümüzde ülkelerin teknolojik bağımsızlığı açısından stratejik öneme sahiptir. Ticari işlemci mimarileri genellikle lisans maliyetleri ve kapalı geliştirme ortamları nedeniyle araştırma ve eğitim faaliyetlerinde çeşitli kısıtlamalar oluşturmaktadır.

Bu projede kullanılan RISC-V mimarisi açık kaynaklı bir komut kümesi mimarisidir. Açık kaynak yapısı sayesinde araştırmacılar ve öğrenciler herhangi bir lisans maliyeti olmadan işlemci mimarisini inceleyebilmekte ve geliştirebilmektedir. Ayrıca PicoRV32 gibi açık kaynaklı işlemci çekirdekleri yerli ve özgün sistemlerin geliştirilmesine katkı sağlamaktadır.

Assembler, linker ve loader modüllerinin öğrenciler tarafından sıfırdan geliştirilmiş olması da ekonomik açıdan önemli bir kazanımdır. Ticari geliştirme araçlarına bağımlılığın azaltılması, araştırma ve geliştirme maliyetlerinin düşürülmesine katkı sağlamaktadır. Bu yaklaşım Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçlarından SKA 8 (İnsana Yakışır İş ve

Ekonomik Büyüme) ve SKA 9 (Sanayi, Yenilikçilik ve Altyapı) hedefleri ile uyumludur. (PÇ8)

5.3 Fonksiyonel Güvenlik ve Sağlık

Gömülü sistemler günümüzde tıbbi cihazlar, otomotiv sistemleri, savunma sanayi uygulamaları ve endüstriyel kontrol sistemleri gibi kritik alanlarda yaygın olarak kullanılmaktadır. Bu sistemlerde meydana gelebilecek yazılım yükleme hataları ciddi güvenlik problemlerine yol açabilmektedir.

Bu proje kapsamında geliştirilen loader sistemi veri bütünlüğünü korumak amacıyla checksum tabanlı hata doğrulama mekanizması kullanılmaktadır. FPGA tarafında gerçekleştirilen doğrulama işlemleri sayesinde bozulmuş veri paketlerinin belleğe yazılması engellenmektedir. Ayrıca işlemci yükleme işlemi tamamlanmadan çalıştırılmamakta ve böylece eksik program yüklemelerinden kaynaklanabilecek hatalar önlenmektedir.

Bu yaklaşım kritik sistemlerde kullanılan güvenli yazılım yükleme prensipleri ile uyumludur. Özellikle tıbbi cihazlar ve güvenlik kritik sistemlerde program bütünlüğünün korunması insan hayatı açısından büyük önem taşımaktadır. Bu nedenle geliştirilen sistem SKA 3 (Sağlık ve Kaliteli Yaşam) hedefleri açısından olumlu katkılar sunmaktadır. (PÇ8)

5.4 E-Atık Yönetimi ve Döngüsel Ekonomi

Elektronik atıklar (e-waste) günümüzde çevresel sürdürülebilirlik açısından önemli sorunlardan biri olarak görülmektedir. Donanımların küçük yazılım değişiklikleri nedeniyle kullanılamaz hale gelmesi elektronik atık miktarını artırmaktadır.

Bu proje kapsamında geliştirilen loader sistemi FPGA üzerinde bulunan yazılımın tekrar tekrar güncellenebilmesine imkan sağlamaktadır. Programların UART üzerinden yeniden yüklenebilmesi sayesinde donanımın fiziksel olarak değiştirilmesine gerek kalmadan yeni özellikler eklenebilmekte veya mevcut hatalar giderilebilmektedir.

Bu yaklaşım donanım ömrünü uzatmakta ve elektronik atık oluşumunu azaltmaktadır. Aynı FPGA platformunun farklı uygulamalar için tekrar kullanılabilmesi döngüsel ekonomi anlayışını desteklemektedir. Böylece mevcut donanım kaynaklarının daha verimli kullanılması sağlanmaktadır. Bu katkılar Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçlarından SKA 12 (Sorumlu Üretim ve Tüketim) ile doğrudan ilişkilidir. (PÇ8)

Sonuç olarak geliştirilen FPGA tabanlı loader sistemi yalnızca teknik açıdan değil, çevresel, ekonomik ve toplumsal açılardan da önemli katkılar sunmaktadır. Açık kaynaklı teknolojilerin kullanılması, enerji verimliliği, güvenli yazılım yükleme mekanizmaları ve elektronik atıkların azaltılmasına yönelik etkileri sayesinde sürdürülebilir mühendislik yaklaşımına uygun bir çözüm ortaya konulmuştur. (PÇ8)

6. PROJE YÖNETİMİ VE TAKIM ÇALIŞMASI

Bu proje, assembler, linker, host loader, UART haberleşme modülü, Loader FSM tasarımı, PicoRV32 entegrasyonu, FPGA doğrulama testleri ve raporlama süreçlerinden oluşan çok modüllü bir mühendislik çalışmasıdır. Projenin başarılı şekilde tamamlanabilmesi için ekip üyeleri arasında görev dağılımı yapılmış, düzenli koordinasyon sağlanmış ve entegrasyon süreçleri sistematik olarak yönetilmiştir. Özellikle donanım ve yazılım bileşenlerinin eş zamanlı geliştirilmesi nedeniyle ekip üyeleri arasında sürekli iletişim kurulmuş ve ortak arayüz standartları belirlenmiştir. Bu yaklaşım proje süresince oluşabilecek entegrasyon problemlerinin erken aşamada tespit edilmesini sağlamıştır. (PÇ13)

6.1 Görev Dağılımı ve Sorumluluk Matrisi

Proje faaliyetleri dört ekip üyesi arasında paylaştırılmıştır. Her üye belirli modüllerin geliştirilmesinden sorumlu olmakla birlikte, sistem entegrasyonu, test süreçleri ve raporlama çalışmaları ekip üyelerinin ortak katılımıyla yürütülmüştür.

Tablo 6.1. Görev Dağılımı ve Sorumluluk Matrisi

Faaliyet	Charaf Eddine Bessanane	Eriñç	Caner Pekkan	Beyza Öner
Literatür Araştırması	✓	✓	✓	✓
Assembler ve Linker Entegrasyonu	✓	✓	✓	✓
Host Loader (Python) Geliştirme	✓	✓		✓
UART Receiver Tasarımı	✓	✓		
Loader FSM Tasarımı	✓	✓		
PicoRV32 ve BRAM Entegrasyonu	✓	✓	✓	
FPGA Testleri ve Doğrulama	✓	✓	✓	
Timing ve Kaynak Analizi	✓	✓	✓	
Rapor Yazımı	✓	✓	✓	
Sunum Hazırlığı	✓	✓	✓	✓

roje kapsamında görev dağılımı ekip üyelerinin teknik yetkinlikleri ve proje ihtiyaçları dikkate alınarak gerçekleştirilmiştir. Host Loader uygulamasının geliştirilmesi, UART haberleşme altyapısının oluşturulması, Loader FSM tasarımı, PicoRV32 işlemcisi ile BRAM entegrasyonu ve sistem seviyesindeki doğrulama çalışmaları ağırlıklı olarak Charaf Eddine Bes ve Erinç Pınar tarafından yürütülmüştür. Caner Pekkan FPGA entegrasyonu, Loader FSM geliştirme süreci ve doğrulama çalışmalarında aktif görev almıştır. Beyza Öner ise literatür araştırması, test süreçleri, raporlama faaliyetleri ve sunum hazırlık çalışmalarına katkı sağlamıştır. Sistem entegrasyonu ve nihai doğrulama çalışmaları tüm ekip üyelerinin ortak katılımıyla gerçekleştirilmiştir. (PÇ13)

6.2 Koordinasyon ve Çalışma Süreci

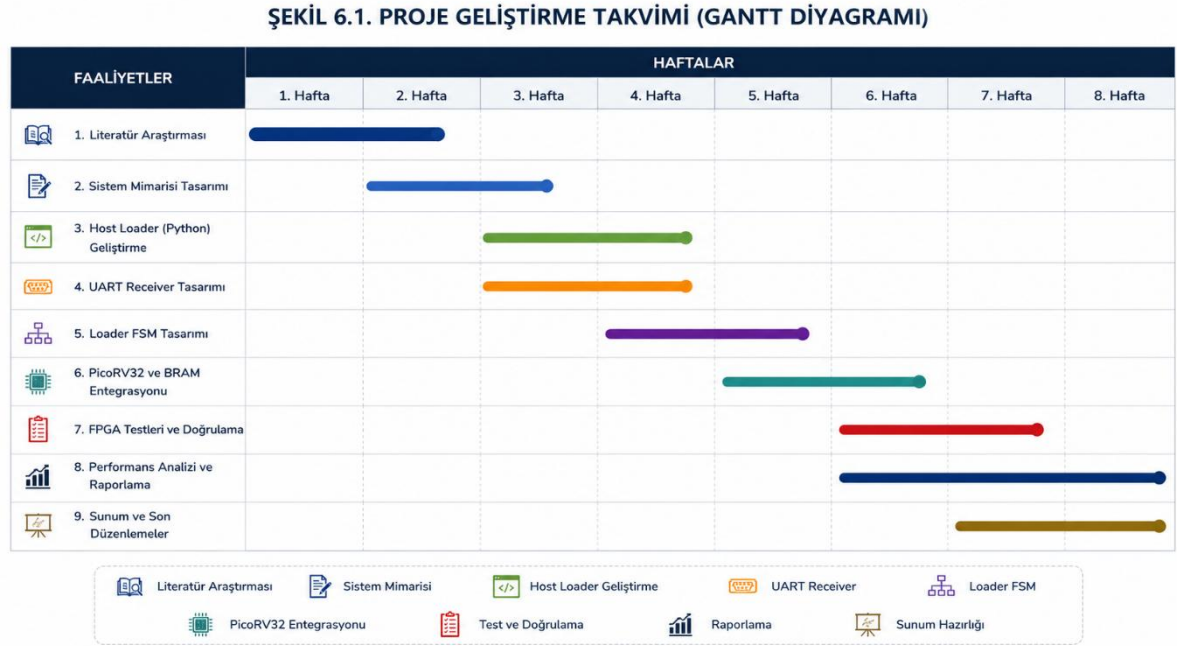
Proje süresince ekip üyeleri düzenli toplantılar gerçekleştirerek geliştirme sürecini takip etmişlerdir. Toplantılarda tamamlanan çalışmalar değerlendirilmiş, karşılaşılan teknik problemler tartışılmış ve sonraki haftalara ait görev planlamaları yapılmıştır. Özellikle assembler, linker ve loader modüllerinin birbirleriyle uyumlu çalışabilmesi için ortak veri formatları ve haberleşme standartları belirlenmiştir. Bu sayede farklı ekip üyeleri tarafından geliştirilen modüllerin entegrasyonu sorunsuz şekilde gerçekleştirilmiştir. (PÇ13)

Tablo 6.2. Toplantı ve İlerleme Takibi

Hafta	Toplantı Konusu	Çıktı
1	Proje planlaması	Görev dağılımı oluşturuldu
2	Literatür araştırması	Loader mimarisi belirlendi
3	UART ve Host Loader	Paket yapısı netleştirildi
4	UART Receiver geliştirme	Haberleşme doğrulandı
5	Loader FSM tasarımı	FSM durumları tamamlandı
6	FPGA entegrasyonu	Modüller birleştirildi
7	Test ve doğrulama	Test planı uygulandı
8	Rapor ve sunum	Son düzenlemeler yapıldı

6.3 Proje Çalışma Takvimi

Şekil 6.1. Proje Geliştirme Takvimi (Gantt Diyagramı)



Şekil 6.1’de proje süresince gerçekleştirilen faaliyetlerin zamansal dağılımı gösterilmektedir. Literatür araştırması ile başlayan süreç, sistem tasarımı, yazılım ve donanım geliştirme çalışmaları, entegrasyon testleri ve raporlama aşamalarının tamamlanmasıyla sonuçlanmıştır. Planlı çalışma takvimi sayesinde proje faaliyetleri belirlenen süre içerisinde tamamlanmıştır. (PÇ13)

7. BİREYSEL KATKI BEYANI

Bu bölüm, proje ekibinde yer alan her öğrenci tarafından ayrı ayrı doldurulmuştur. Amaç, ekip çalışması kapsamında gerçekleştirilen faaliyetler içerisinde her öğrencinin bireysel katkılarını, sorumluluk aldığı modülleri ve karşılaştığı teknik problemleri ortaya koymaktır. Bu kapsamda öğrenciler geliştirdikleri modülleri, üstlendikleri görevleri ve çözüm ürettikleri teknik problemleri aşağıda detaylı olarak açıklamışlardır. (PÇ12)

7.1 Charaf Eddine Bes

Bu proje kapsamında ağırlıklı olarak Host Loader uygulamasının geliştirilmesi, Loader FSM tasarımı, PicoRV32 işlemcisi ile BRAM entegrasyonu ve sistem seviyesindeki doğrulama çalışmalarında görev aldım. Özellikle Python dili kullanılarak geliştirilen Host Loader uygulamasının tasarımında aktif rol üstlendim. Bu uygulama linker tarafından oluşturulan .mem dosyasını okuyarak UART üzerinden FPGA kartına aktarılacak veri paketlerini oluşturmakta, checksum hesaplamalarını gerçekleştirmekte ve yükleme sürecini yönetmektedir.

Ayrıca Loader FSM modülünün geliştirilmesi sürecinde görev alarak UART üzerinden gelen paketlerin alınması, checksum doğrulaması, BRAM belleğine yazılması ve işlemcinin yükleme sonrasında çalıştırılması süreçlerinin tasarlanmasına katkı sağladım. FPGA entegrasyon aşamasında PicoRV32 işlemcisi, BRAM yapısı ve Loader FSM modüllerinin birlikte çalışmasını sağlayan doğrulama çalışmalarında aktif olarak yer aldım.

Karşılaştığım en önemli teknik problem, UART üzerinden gönderilen veri paketlerinin FPGA tarafında güvenilir şekilde işlenmesi ve işlemcinin yükleme tamamlanmadan çalışmasının engellenmesiydi. Bu problemi Loader FSM tasarımında yapılan düzenlemeler, checksum doğrulama mekanizması ve bellek yönetim mantığının iyileştirilmesi ile çözdüm. Böylece sistemin güvenilir şekilde çalışması sağlandı. (PÇ12)

7.2 Erinç Pırıldar

Bu proje kapsamında UART haberleşme altyapısının geliştirilmesi, Host Loader uygulamasının geliştirilmesi, Loader FSM tasarımı ve FPGA doğrulama çalışmalarında aktif görev aldım. Özellikle UART Receiver modülünün tasarımı, seri haberleşme mekanizmasının FPGA üzerinde güvenilir şekilde çalıştırılması ve yükleme protokolünün oluşturulması süreçlerinde sorumluluk üstlendim.

Loader FSM geliştirme çalışmalarında görev alarak veri alma, checksum doğrulama, bellek yazma ve işlemci kontrol mekanizmalarının oluşturulmasına katkı sağladım. Ayrıca FPGA üzerinde gerçekleştirilen entegrasyon ve doğrulama çalışmalarında yer alarak sistemin farklı test senaryoları altında davranışlarını analiz ettim ve hata ayıklama süreçlerine katıldım.

Karşılaştığım en önemli teknik problem, UART haberleşmesinde oluşabilecek senkronizasyon problemlerinin önlenmesi ve veri bütünlüğünün korunmasıydı. Bu problemi senkronizasyon baytı, checksum doğrulama mekanizması ve UART

alıcı tasarımımda gerçekleştirilen iyileştirmeler ile çözdüm. Yapılan testler sonucunda haberleşme altyapısının güvenilir şekilde çalıştığı doğrulandı. (PÇ12)

7.3 Caner Pekkan

Bu proje kapsamında Loader FSM geliştirme çalışmaları, PicoRV32 entegrasyonu ve FPGA doğrulama süreçlerinde görev aldım. Özellikle sistem modüllerinin FPGA üzerinde birlikte çalıştırılması ve test edilmesi aşamalarında aktif rol üstlendim.

Doğrulama sürecinde farklı test senaryoları uygulanarak sistem davranışları analiz edilmiş, bellek erişimleri ve program yükleme süreçleri incelenmiştir. Ayrıca kaynak kullanım raporları ve zamanlama analizlerinin değerlendirilmesine katkı sağladım.

Karşılaştığım en önemli teknik problem, farklı modüllerin entegrasyonu sırasında oluşan haberleşme ve zamanlama uyumsuzluklarının giderilmesiydi. Yapılan entegrasyon testleri ve hata analizleri sonucunda bu problemler çözüme kavuşturulmuş ve sistemin kararlı şekilde çalışması sağlanmıştır. (PÇ12)

7.4 Beyza Öner

Bu proje kapsamında literatür araştırması, test süreçleri, raporlama faaliyetleri ve sunum hazırlık çalışmalarında görev aldım. Özellikle FPGA tabanlı loader sistemleri, UART haberleşmesi, veri doğrulama yöntemleri ve RISC-V mimarisi üzerine literatür araştırmaları gerçekleştirerek proje dokümantasyonuna katkı sağladım.

Ayrıca sistem doğrulama sürecinde gerçekleştirilen testlerin raporlanması ve elde edilen sonuçların dokümanite edilmesi çalışmalarında görev aldım. Proje raporunun hazırlanması ve sunum materyallerinin oluşturulması aşamalarında aktif katkı sağladım.

Karşılaştığım en önemli teknik problem, farklı kaynaklardan elde edilen bilgilerin proje gereksinimleri ile uyumlu hale getirilmesiydi. Yapılan literatür incelemeleri ve ekip içi değerlendirmeler sonucunda gerekli bilgiler filtrelenmiş ve proje kapsamında kullanılabilir hale getirilmiştir. (PÇ12)

8. SONUÇ

Bu projede, önceki aşamalarda geliştirilen assembler ve linker sistemleri FPGA üzerinde çalışan özgün bir loader mimarisi ile entegre edilerek uçtan uca çalışan bir RISC-V geliştirme ortamı oluşturulmuştur. Geliştirilen sistem sayesinde assembly kaynak kodları assembler tarafından object dosyalarına dönüştürülmüş, linker tarafından birleştirilmiş ve elde edilen makine kodları UART tabanlı haberleşme altyapısı üzerinden FPGA kartına aktarılmıştır.

Proje kapsamında bilgisayar tarafında çalışan Host Loader uygulaması geliştirilmiş, veri bütünlüğünü sağlamak amacıyla checksum tabanlı hata kontrol mekanizması kullanılmıştır. FPGA tarafında ise UART Receiver modülü, Loader FSM yapısı, BRAM tabanlı program belleği ve PicoRV32 işlemcisi entegre edilmiştir. Loader FSM, veri alma, paket doğrulama, bellek yazma ve işlemciyi çalıştırma süreçlerini yöneten merkezi kontrol birimi olarak tasarlanmıştır.

Sistemin doğrulanması amacıyla farklı senaryolar altında kapsamlı testler gerçekleştirilmiştir. UART haberleşmesi, checksum doğrulaması, bellek yazma işlemleri, adres yönetimi, MMIO erişimleri, zamanlama analizi ve sistem entegrasyonu testleri başarıyla tamamlanmıştır. Ayrıca sentez raporlarında elde edilen kaynak kullanım değerleri ve pozitif timing slack sonuçları sistemin hedeflenen 27 MHz çalışma frekansında güvenilir şekilde çalıştığını göstermiştir.

Proje sonunda assembler, linker, loader, UART haberleşmesi, FPGA tabanlı bellek yönetimi ve RISC-V işlemci mimarisi tek bir sistem içerisinde başarıyla birleştirilmiştir. Böylece bilgisayar mimarisi, sistem programlama ve donanım-yazılım ortak tasarımı alanlarında önemli mühendislik deneyimleri kazanılmıştır.

Gelecekte yapılabilecek çalışmalar kapsamında checksum yerine CRC tabanlı hata kontrol mekanizmasının kullanılması, SPI veya Ethernet tabanlı daha yüksek hızlı yükleme protokollerinin eklenmesi, uzaktan güncelleme (OTA) desteğinin sağlanması ve daha büyük RISC-V işlemci çekirdekleri ile entegrasyonun gerçekleştirilmesi planlanmaktadır.

(PÇ6, PÇ7, PÇ8, PÇ12, PÇ13)

KAYNAKÇA

- [1] Patterson, D. A., & Hennessy, J. L. (2021). Computer Organization and Design RISC-V Edition: The Hardware Software Interface (2nd Edition). Morgan Kaufmann.
- [2] Wolf, C. (2024). PicoRV32 - A Size Optimized RISC-V CPU. GitHub Repository. <https://github.com/YosysHQ/picorv32>
- [3] RISC-V International. (2024). The RISC-V Instruction Set Manual Volume I: Unprivileged ISA. RISC-V Foundation.
- [4] IEEE Standards Association. (2021). IEEE Standard for Information Technology and Telecommunications.
- [5] NIST. (2022). Platform Firmware Resiliency Guidelines (SP 800-193). National Institute of Standards and Technology.
- [6] JEDEC Solid State Technology Association. (2023). JESD Standard for Semiconductor Memory Devices.
- [7] RTCA. (2020). Design Assurance Guidance for Airborne Electronic Hardware (DO-254).
- [8] Stallings, W. (2019). Computer Organization and Architecture: Designing for Performance (11th Edition). Pearson.
- [9] Mano, M. M., & Ciletti, M. D. (2018). Digital Design with RTL Design, Verilog and VHDL. Pearson.
- [10] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th Edition). Wiley.
- [11] Tanenbaum, A. S., & Bos, H. (2015). Modern Operating Systems (4th Edition). Pearson.
- [12] GNU Project. (2024). GNU Assembler (GAS) Documentation. GNU Organization.
- [13] LLVM Project. (2024). LLVM Compiler Infrastructure Documentation.
- [14] NASM Development Team. (2024). NASM - The Netwide Assembler Documentation.
- [15] Xilinx Inc. (2024). FPGA Design Flow User Guide.